

**TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN
ĐẠI HỌC QUỐC GIA TP HỒ CHÍ MINH**



**BÁO CÁO ĐỒ ÁN
THIẾT KẾ ỨNG DỤNG: TRÒ CHƠI ĐẤY HỘP
KẾT HỢP YẾU TỐ ĐỐI KHÁNG**

—o0o—

Sinh viên thực hiện: Lê Phú Trọng

Mã số sinh viên: 25521937

Mã môn học: IT003.Q21.CTTN

Giáo viên hướng dẫn: Thầy Nguyễn Thanh Sơn

Mục lục

1	Giới thiệu	1
1.1	Đặt vấn đề	1
1.2	Giới thiệu về Sokoban và nguồn gốc bài toán	1
1.3	Lý do chọn đề tài	2
1.4	Giải pháp hiện tại	2
1.5	Giải pháp của tác giả	2
1.6	Đóng góp chính	3
2	Mô tả chung	4
2.1	Mô tả chung về ứng dụng	4
2.2	Luật chơi chi tiết	4
2.3	Một số Cấu trúc dữ liệu và Giải thuật được sử dụng	5
3	Chi tiết các nghiên cứu về các cấu trúc dữ liệu và giải thuật	7
3.1	Thuật toán sinh bản đồ	7
3.2	Thuật toán AI	8
3.2.1	Thuật toán của tác giả	8
3.2.2	Chiến lược chặn đường	9
3.2.3	Thuật toán Beam Search	10
3.2.4	Các chế độ AI	11
4	Thực nghiệm	12
4.1	Quá trình thực hiện	12
4.1.1	Tuần 1	12
4.1.2	Tuần 2	12
4.1.3	Tuần 3	12
4.1.4	Tuần 4	12
4.2	Bảng tự phân công	12
4.3	Phân tích kết quả thực nghiệm	13
4.3.1	Đánh giá tổng quan năng lực	13
4.3.2	Ảnh hưởng của kích thước bản đồ	14
4.3.3	Tác động của chế độ bản đồ và giới hạn số nước đi	15
4.3.4	Kết luận	15
5	Tổng kết	16
5.1	Kết quả đạt được	16
5.2	Hạn chế	16
5.3	Hướng phát triển trong tương lai	16
	Tài liệu tham khảo	18
A	Phụ lục	19
A.1	Giới thiệu (demo) kết quả	19
A.1.1	Màn hình chính	19
A.1.2	Màn hình hướng dẫn	19
A.1.3	Màn hình thiết lập	20
A.1.4	Màn hình trò chơi	20
A.2	Kết quả thực nghiệm	21
A.2.1	Thiết lập thực nghiệm	21



A.2.2	Bảng kết quả chi tiết	21
A.3	Chi tiết mã nguồn	24
A.3.1	Hướng dẫn triển khai	24
A.3.2	Cấu trúc thư mục	24
A.3.3	Docstring	25

1 Giới thiệu

1.1 Đặt vấn đề

Trò chơi điện tử (Game) từ lâu đã không còn là một loại hình giải trí xa lạ mà đã trở thành một phần không thể thiếu trong đời sống văn hóa và tinh thần của con người hiện đại. Với sự phát triển không ngừng của công nghệ, thế giới trò chơi đã tiến hóa từ những khối pixel đơn giản đến những môi trường mô phỏng phức tạp, mang lại những trải nghiệm tương tác đa dạng. Tuy nhiên, đằng sau những giây phút giải trí đó là những thách thức không nhỏ đối với các nhà phát triển trong việc xử lý logic hệ thống và tối ưu hóa trải nghiệm người dùng. Thực tế cho thấy việc thiết kế một trò chơi vừa đảm bảo tính giải trí, vừa có nền tảng thuật toán vững chắc để vận hành mượt mà trong thời gian thực.

Hiện nay, thị trường trò chơi điện tử chủ yếu được dẫn dắt bởi các dòng game cạnh tranh và đối kháng theo thời gian thực như League of Legends, PUBG hay Valorant. Trong khi đó, các trò chơi giải đố tuy vẫn có cộng đồng người chơi riêng nhưng nhìn chung không duy trì được sức hút mạnh mẽ. Đối với Sokoban, hiện đã có nhiều phiên bản và biến thể khác nhau. Tuy nhiên, phần lớn vẫn tập trung vào trải nghiệm giải đố đơn người chơi hoặc hợp tác nhiều người chơi để vượt màn. Các biến thể mang tính đối kháng còn tương đối hiếm. Trong khi các màn chơi thường được thiết kế thủ công nên tính mới mẻ không cao sau khi người chơi đã tìm ra lời giải tối ưu. Vì vậy, việc kết hợp yếu tố giải đố của Sokoban với tính cạnh tranh của dòng game đối kháng có thể tận dụng được sức hút của dòng game đối kháng, từ đó lan tỏa những giá trị của dòng game giải đố truyền thống.

1.2 Giới thiệu về Sokoban và nguồn gốc bài toán

Sokoban là một trò chơi giải đố cổ điển được Hiroyuki Imabayashi phát triển vào năm 1982 [Imabayashi, 1982], Wikipedia contributors [2024]. Trong trò chơi này, người chơi phải di chuyển và đẩy các khối hộp đến những vị trí mục tiêu trong một không gian giới hạn. Dù luật chơi tương đối đơn giản, Sokoban lại nổi tiếng bởi độ khó cao do yêu cầu lập kế hoạch dài hạn và số lượng trạng thái rất lớn. Do đó, việc đưa ra lời giải cho trò chơi này đã trở thành một bài toán khó trong lĩnh vực nghiên cứu về giải thuật.

Tại Việt Nam, ý tưởng bài toán này cũng đã từng xuất hiện ở đề thi Chung kết Tin học trẻ Toàn quốc Bảng C1 năm 2025 Hệ thống thi Tin học trẻ Toàn quốc [2025]. Tại đó, bài toán được đặt ra cho các thí sinh là thiết kế AI giải trò chơi đẩy hộp. Tuy nhiên, bài toán được tổ chức dưới hình thức đối kháng, trong đó hai thí sinh cùng nhau chơi trên một bàn cờ trong một số lượt. Khi kết thúc, điểm của mỗi thí sinh sẽ được tính, từ đó đưa ra kết quả chung cuộc. Tác giả cũng từng là một thí sinh đã có cơ hội tiếp cận sâu sắc với bài toán này. Đây là một cột mốc quan trọng khi tác giả nhận được sự hỗ trợ nhiệt tình từ bạn bè, thầy cô ở trường phổ thông và thậm chí là sự đồng hành từ phía Trường Đại học Công nghệ Thông tin (UIT) trong thời gian diễn ra trại hè UIT BootCamp. Việc UIT đã cung cấp phương tiện di chuyển và tạo điều kiện thuận lợi cho các thí sinh tham dự vòng loại khu vực miền Nam đã tạo động lực lớn để tác giả có thể vượt qua vòng loại này, tiếp tục tham dự vòng chung kết và có cơ hội tiếp cận với bài toán và nghiên cứu sâu hơn.

Thông qua quá trình tìm hiểu, tham khảo, nghiên cứu và thực nghiệm, tác giả nhận thấy bài toán này vẫn còn nhiều tiềm năng mở rộng, đặc biệt khi chuyển từ môi trường giải đố tĩnh sang môi trường có tính cạnh tranh và tương tác thời gian thực giữa nhiều người chơi.

1.3 Lý do chọn đề tài

Bên cạnh yếu tố cảm hứng, lý do quan trọng hơn dẫn đến việc lựa chọn đề tài này chính là tính ứng dụng cao của các kiến thức của môn Cấu trúc dữ liệu và Giải thuật (DSA). Qua quá trình tìm hiểu và nghiên cứu lâu dài, tác giả nhận thấy bài toán tại vòng chung kết Tin học trẻ năm đó chứa đựng nhiều "không gian" để khai thác và tối ưu hóa. Việc đạt thành tích Giải Nhì (Top 3) toàn quốc là một minh chứng cho sự ổn định của các giải thuật nền tảng, nhưng đồng thời cũng đặt ra nhu cầu phải xây dựng những thuật toán mạnh mẽ hơn để đáp ứng luật chơi mới. Chính vì vậy, đề tài “Thiết kế ứng dụng: Trò chơi đẩy hộp kết hợp yếu tố đối kháng” đã được lựa chọn để thực hiện đồ án môn học IT003.Q21.CTTN.

Mục tiêu của đề tài không chỉ dừng lại ở việc xây dựng hay tái hiện một trò chơi giải trí, mà còn hướng đến việc mở rộng bài toán Sokoban truyền thống sang môi trường đối kháng thời gian thực có không gian trạng thái lớn hơn và mức độ tương tác phức tạp hơn. Điều này đặt ra nhu cầu nghiên cứu các hướng tiếp cận phù hợp nhằm cân bằng giữa chất lượng đánh giá trạng thái và tốc độ xử lý trong thời gian thực.

Thông qua đồ án, tác giả có cơ hội tiếp tục thử nghiệm và tinh chỉnh các giải thuật tìm kiếm, heuristic evaluation và chiến lược xử lý trạng thái trong môi trường động nhiều tác nhân, vốn là những vấn đề mang tính đặc trưng trong lĩnh vực Cấu trúc dữ liệu và Giải thuật.

1.4 Giải pháp hiện tại

Về mặt giải thuật, hiện nay bài toán Sokoban và các biến thể của nó thường được giải quyết thông qua hai hướng tiếp cận chính: tìm kiếm cổ điển và học máy. Trong đó, các thuật toán tìm kiếm như A*, IDA* hay BFS vẫn là nền tảng phổ biến nhờ khả năng đảm bảo tính đúng đắn và tối ưu lời giải khi được trang bị các heuristic phù hợp. Bên cạnh đó, các phương pháp như Monte Carlo Tree Search (MCTS) cũng được sử dụng để xử lý các tình huống có độ bất định cao, đặc biệt khi không thể xây dựng heuristic hiệu quả. Tuy nhiên, do luật chơi được thay đổi và cải tiến, việc có thêm nhiều tác nhân ảnh hưởng đến trò chơi mà chủ yếu là đến từ đối thủ khiến các phương pháp như MCTS không còn hiệu quả khi không thể đánh giá khách quan được toàn diện bản đồ. Ngoài ra, các thuật toán tìm kiếm toàn cục như Minimax hoặc MCTS cũng gặp hạn chế khi branching factor tăng nhanh theo số lượng tác nhân và kích thước bản đồ.

Song song với đó, các phương pháp học máy, đặc biệt là Deep Reinforcement Learning (DRL), đã được nghiên cứu nhằm cho phép tác nhân tự học chiến lược giải quyết bài toán thông qua tương tác với môi trường. Tuy nhiên, do đặc thù của Sokoban là có không gian trạng thái lớn và yêu cầu lập kế hoạch dài hạn, các phương pháp RL truyền thống thường gặp khó khăn. Vì vậy, xu hướng gần đây tập trung vào Hierarchical Reinforcement Learning (HRL) Pastukhov [2025]. Tuy nhiên, việc triển khai một mô hình học tăng cường lớn trong khi vẫn phải đòi hỏi tốc độ xử lý là một yêu cầu tương đối bất khả thi. Do đó, việc xây dựng các heuristic hiệu quả với chi phí tính toán thấp trở thành một hướng tiếp cận phù hợp hơn đối với môi trường Sokoban đối kháng thời gian thực.

1.5 Giải pháp của tác giả

Trong phạm vi đồ án này, tác giả đề xuất một hướng tiếp cận kết hợp giữa trò chơi giải đồ Sokoban và yếu tố đối kháng thời gian thực. Trò chơi được xây dựng theo mô hình hai người chơi cùng tương tác trên một bản đồ chung, thay đổi một số luật gốc của trò chơi, từ đó biến bài toán Sokoban truyền thống từ bài toán tĩnh thành một hệ thống động có tính cạnh tranh cao. Người chơi không chỉ cần tối ưu thao tác đẩy hộp mà còn phải phản ứng với hành động của đối thủ để giành lợi thế.

Để giải quyết bài toán đó, đề án tập trung xây dựng hệ thống quản lý trạng thái hiệu quả nhằm đảm bảo việc cập nhật vị trí người chơi, hộp và các đối tượng trên bản đồ diễn ra nhanh chóng và chính xác. Ngoài ra, hệ thống sinh bản đồ ngẫu nhiên đối xứng cũng được triển khai nhằm tăng tính đa dạng của trò chơi và đảm bảo sự công bằng giữa hai người chơi.

Đồng thời, trò chơi cũng hỗ trợ cơ chế đấu với máy nhằm giúp người chơi có thể luyện tập ngay cả khi không có đối thủ thực. Trong đó, việc xây dựng hệ thống AI có khả năng phản hồi nhanh và ổn định trong môi trường thời gian thực chính là trọng tâm nghiên cứu của đề án. Để đáp ứng yêu cầu đó, đề án ưu tiên sử dụng các heuristic đơn giản nhưng hiệu quả thay vì các thuật toán tìm kiếm có chi phí tính toán lớn như Minimax hay Monte Carlo Tree Search. Các thuật toán tìm đường đi nền tảng như BFS và Dijkstra được áp dụng để xử lý các bài toán di chuyển và đẩy hộp cơ bản, trong khi Beam Search được nghiên cứu nhằm mở rộng khả năng đánh giá nhiều trạng thái tiềm năng cùng lúc. Nhờ đó, hệ thống AI có thể đưa ra quyết định với độ ổn định tương đối trong khoảng thời gian xử lý ngắn.

Thông qua các giải pháp trên, đề án hướng tới việc xây dựng một trò chơi hoàn chỉnh có tính cạnh tranh, khả năng mở rộng và vận hành ổn định. Đồng thời tạo môi trường để thử nghiệm các chiến thuật và giải thuật trong bài toán Sokoban đối kháng, vốn cũng là mục tiêu cốt lõi mà tác giả hướng tới trong quá trình thực hiện đề án môn học Cấu trúc dữ liệu và Giải thuật.

1.6 Đóng góp chính

Trong phạm vi đề án này, các đóng góp chính của tác giả bao gồm:

- Xây dựng một biến thể Sokoban đối kháng thời gian thực cho hai người chơi trên cùng một bản đồ, mở rộng bài toán Sokoban truyền thống từ môi trường tĩnh sang môi trường động có tính cạnh tranh.
- Thiết kế các cơ chế xử lý hành động đồng thời và va chạm giữa các tác nhân nhằm hỗ trợ các cơ chế như đẩy nhiều hộp, tranh chấp vị trí và tương tác trực tiếp giữa hai người chơi.
- Xây dựng hệ thống sinh bản đồ ngẫu nhiên đối xứng với nhiều chế độ khác nhau nhằm tăng tính đa dạng và đảm bảo tính công bằng trong quá trình thi đấu.
- Phát triển hệ thống AI với nhiều mức độ khó dựa trên các thuật toán BFS, Dijkstra, heuristic evaluation và Beam Search nhằm đáp ứng yêu cầu phản hồi nhanh trong môi trường thời gian thực.
- Thực hiện thực nghiệm trên nhiều cấu hình bản đồ và giới hạn bước đi khác nhau để đánh giá chất lượng và mức độ ổn định của các chiến lược AI được đề xuất.

2 Mô tả chung

2.1 Mô tả chung về ứng dụng

Đồ án này là một phiên bản biến thể từ trò chơi Sokoban truyền thống. Trò chơi này có thể được chơi từ 1 đến 2 người. Trong đó, luật chơi được thêm vào yếu tố đối kháng để có thể chơi nhiều người, đồng thời cũng được thay đổi ở vài điểm để phù hợp với luật chơi đối kháng và gia tăng trải nghiệm game. Ví dụ, trong trò chơi gốc, đẩy hộp vào ô đích không làm mất hộp đó, có nhiều ô đích và phải đẩy tất cả hộp vào tất cả ô đích để chiến thắng, trong trò chơi này, đẩy hộp vào ô đích sẽ tăng điểm của người chơi và làm hộp đó biến mất. Cơ chế đẩy nhiều hộp (Chain pushing) cũng là một điểm thay đổi của luật chơi. Mọi thông tin chi tiết hơn về luật chơi vui lòng tham khảo mục 2.2.

Bên cạnh cơ chế chơi cốt lõi, ứng dụng còn hỗ trợ nhiều tính năng mở rộng nhằm nâng cao trải nghiệm người dùng. Cụ thể, trò chơi cho phép sinh bản đồ một cách linh hoạt với các thiết lập tùy chỉnh theo ý muốn của người chơi, bao gồm kích thước bản đồ, số lượng hộp, cũng như vị trí các đối tượng trong môi trường. Ngoài ra, hệ thống còn cung cấp chế độ chơi với AI được thiết kế với nhiều mức độ khó khác nhau, giúp người chơi có thể luyện tập hoặc thi đấu ngay cả khi không có đối thủ thực. Bên cạnh đó, ở màn hình trò chơi còn hỗ trợ một số nút chức năng giúp trải nghiệm game của người chơi tốt hơn.

Về mặt triển khai, đồ án được xây dựng bằng sự kết hợp giữa hai ngôn ngữ lập trình Python và C++, trong đó Python đảm nhiệm vai trò chính trong việc xử lý logic trò chơi và giao diện, còn C++ được sử dụng để tối ưu hóa một số thành phần yêu cầu hiệu năng cao. Giao diện đồ họa của trò chơi được phát triển dựa trên thư viện Pygame Shinnars [2000], cho phép hiển thị môi trường trò chơi một cách trực quan và tương tác theo thời gian thực.

Người chơi có thể điều khiển nhân vật thông qua bàn phím để di chuyển. Trong đó, các phím WASD được dùng để di chuyển người chơi thứ nhất và các phím mũi tên được dùng để di chuyển người chơi thứ hai. Đồng thời, người chơi cũng có thể sử dụng chuột để thao tác với các nút chức năng hỗ trợ như thiết lập cấu hình trò chơi và các nút hỗ trợ có sẵn. Nhờ đó, ứng dụng không chỉ đảm bảo tính chính xác về mặt thuật toán mà còn mang lại trải nghiệm sử dụng thân thiện và linh hoạt.

2.2 Luật chơi chi tiết

Trong bản đồ của trò chơi, tồn tại các loại đối tượng sau:

- Ô trống (Empty): Ô trống, người chơi có thể di chuyển vào ô này hoặc đẩy hộp vào ô này. Ô này được mã hóa thành ký tự .
- Ô vật cản (Obstacle): Chứa một vật cản, người chơi không thể di chuyển vào ô này cũng như không thể đẩy hộp vào ô này. Ô này được mã hóa thành ký tự #
- Ô hộp (Box): Chứa một hộp, hộp trong ô này có thể được di chuyển sang ô trống kề cạnh bằng cách bị một người chơi đẩy vào. Hộp cũng có thể được đẩy vào ô đích. Khi hộp được đẩy vào một ô đích, hộp biến mất và người chơi sở hữu ô đích đó được tăng một điểm. Ô này được mã hóa thành ký tự X
- Ô đích (Goal): Chứa một ô đích của một trong hai người chơi. Ô này chỉ xuất hiện duy nhất một lần trong cả bản đồ đối với mỗi ô đích của người chơi. Ô này được mã hóa thành ký tự A đối với ô đích của người chơi 1 và B đối với ô đích của người chơi 2.
- Ô nhân vật (Character): Chứa một nhân vật của một trong hai người chơi. Ô này chỉ xuất hiện duy nhất một lần trong cả bản đồ đối với mỗi người chơi. Ô này được mã hóa thành

ký tự a đối với người chơi 1 và b đối với người chơi 2.

Trò chơi diễn ra trong T lượt. Mỗi lượt, cả hai người chơi đồng thời điều khiển nhân vật của mình di chuyển. Khi người chơi đẩy hộp vào một ô đích, thì người chơi sở hữu ô đích đó sẽ được tăng một điểm. Sau khi kết thúc T lượt, người chơi có số điểm cao hơn giành chiến thắng. Trong trường hợp số điểm của cả hai người chơi hòa nhau, người chơi ghi điểm cuối cùng trước sẽ giành chiến thắng.

Các tương tác cụ thể:

- Khi người chơi đứng ở ô chung cạnh với hộp và di chuyển về phía của hộp, hộp sẽ bị đẩy theo hướng di chuyển của người chơi.
- Người chơi có thể đẩy nhiều hộp cùng một lúc. Đây là luật đặc biệt được thay đổi riêng trong đề án này.
- Người chơi không thể di chuyển hay đẩy hộp vào ô vật cản.
- Người chơi có thể đẩy hộp vào ô đích nhưng không thể di chuyển vào ô đích.
- Khi một hộp được đẩy vào ô đích thì người chơi sở hữu ô đích đó sẽ được cộng điểm.

Các tương tác đặc biệt: Do cả hai người chơi được xử lý đồng thời, một số trường hợp xung đột có thể xảy ra. Dưới đây là một số kết quả cho các xung đột:

- Hai người chơi không thể đứng ở cùng một ô. Nếu cả hai cùng di chuyển vào cùng một ô trống, lượt điều khiển của cả hai người chơi sẽ thất bại và cả hai người chơi sẽ đứng yên.
- Hai người chơi không thể đẩy cùng 1 hộp. Khi hai người chơi cùng thực hiện đẩy và tác động lên cùng một hộp, lượt điều khiển của cả hai người chơi sẽ thất bại và cả hai người chơi sẽ đứng yên.
- Nếu người chơi đẩy hộp vào ô mà đối thủ cũng đi vào, đối thủ được ưu tiên, hành động của người chơi thất bại.
- Nếu một người chơi đẩy hộp vào ô mà đối thủ đang đứng, nhưng trong cùng một lượt đó đối thủ di chuyển khỏi ô đang đứng, hành động của người chơi vẫn thành công.
- Khi hai người chơi đứng cạnh nhau và cùng di chuyển về hướng của đối thủ, vị trí của hai người chơi sẽ được hoán đổi cho nhau.

2.3 Một số Cấu trúc dữ liệu và Giải thuật được sử dụng

Hệ thống đã kết hợp linh hoạt nhiều cấu trúc dữ liệu và giải thuật từ cơ bản đến nâng cao nhằm tối ưu hóa cả trải nghiệm người dùng lẫn hiệu năng của trí tuệ nhân tạo. Dưới đây là danh sách một số cấu trúc dữ liệu và giải thuật đã được ứng dụng:

- Mã hóa: Về mặt quản lý dữ liệu, đề án sử dụng kỹ thuật Mã hóa để ánh xạ các trạng thái trò chơi và lệnh điều khiển thành các ký tự hoặc con số, giúp tối ưu quá trình truyền tải dữ liệu giữa các thành phần hệ thống. Cụ thể, bản đồ bao gồm các đối tượng của trò chơi được mã hóa thành các ký tự khác nhau được nêu ở mục 2.2 để các chương trình con giao tiếp với nhau và với hệ thống trí tuệ nhân tạo.
- Stack: Đóng vai trò then chốt trong việc xử lý logic vật lý, cho phép nhân vật thực hiện hành động đẩy nhiều khối hộp cùng lúc một cách chính xác. Đồng thời hỗ trợ cho cơ chế hỗ trợ hoàn tác lượt chơi.
- Hash Table (triển khai qua Python dictionary): Được tận dụng để lưu trữ toàn bộ thông tin về trạng thái trò chơi, phục vụ việc truy xuất nhanh chóng khi tương tác với bộ interactor. Do các thông tin của trò chơi không chỉ có bản đồ mà còn có điểm số của hai người chơi,



lịch sử, việc sử dụng cấu trúc dữ liệu này đảm bảo được tốc độ xử lý, sự gọn gàng và dễ hiểu trong code khi các chức năng của phần mềm tương tác với nhau.

- Random: Phục vụ cho mục đích sinh bản đồ ngẫu nhiên ở mỗi lần chơi, đồng thời dùng để mô phỏng tương tác của bot. Chi tiết hơn về thuật toán sinh bản đồ vui lòng tham khảo 3.1
- Toán: Dùng để tinh chỉnh các đối tượng trong giao diện đồ họa.
- BFS/DFS, Dijkstra, Beam search: Qua quá trình thử nghiệm nhiều thuật toán, đây là các thuật toán đã được chọn. Một số lý do những thuật toán này được chọn sẽ được đề cập cụ thể hơn ở mục 3.2. Heuristic: Sử dụng trong AI của trò chơi, dùng để đánh giá các trạng thái của trò chơi, từ đó giúp mô hình đưa ra các quyết định tối ưu.

3 Chi tiết các nghiên cứu về các cấu trúc dữ liệu và giải thuật

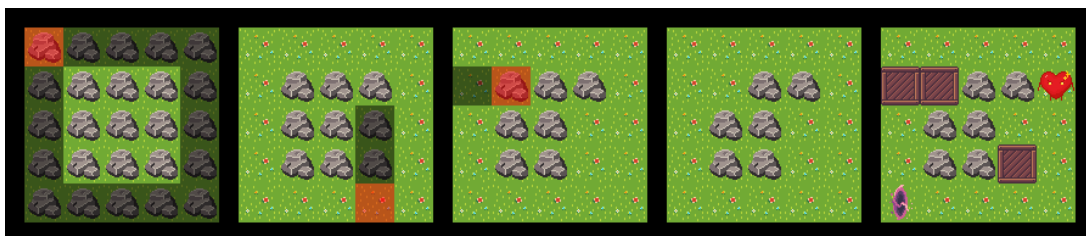
3.1 Thuật toán sinh bản đồ

Quá trình sinh bản đồ có mục tiêu là đảm bảo tính đa dạng nhưng vẫn giữ được sự cân bằng giữa hai người chơi. Cụ thể, chương trình trước hết khởi tạo một ma trận kích thước $\frac{N}{2} \times N$ và tiến hành sinh nửa trên của bản đồ bằng các chiến lược ngẫu nhiên có kiểm soát, sau đó đối xứng hóa xuống nửa dưới. Việc sử dụng bộ sinh số ngẫu nhiên kết hợp với các hàm sinh có trọng số (weighted random) cho phép điều chỉnh phân bố của các vật cản và cấu trúc mê cung, từ đó tạo ra nhiều dạng bản đồ khác nhau mà vẫn tránh các trường hợp quá dễ hoặc quá khó.

Hàm sinh có trọng số (weighted random) có 3 biến đầu vào là k, l, r , lần lượt là trọng số ngẫu nhiên và khoảng ngẫu nhiên. $sgn(k)$ cho biết kết quả có thiên hướng lệch trái (âm) hay phải (dương), trong khi đó $|k|$ cho biết trọng số, hay nói cách khác là mức độ lệch của kết quả. Cụ thể, hàm weighted random có kết quả như sau:

$$wRand(k, l, r) = \begin{cases} Rand(l, r), & k = 0, \\ \min(Rand(l, r), wRand(k - 1, l, r)), & k > 0, \\ \max(Rand(l, r), wRand(k - 1, l, r)), & \text{otherwise} \end{cases} \quad (1)$$

Đối với từng chế độ bản đồ, quá trình sinh có sự khác biệt rõ rệt. Với chế độ NON, bản đồ được tạo hoàn toàn không có vật cản. Với chế độ SEP, chương trình sinh hai nửa bản đồ riêng biệt và chèn thêm một hàng vật cản ở giữa để ngăn cách hoàn toàn hai người chơi, tạo nên hai không gian độc lập. Trong khi đó, chế độ SYM tạo ra một bản đồ ngẫu nhiên tiêu chuẩn, gồm có các hộp cũng như các vật cản.



Hình 1: Minh họa cơ chế sinh bản đồ. Trong 3 bước đầu, các ô được tô đỏ là các ô được chọn làm gốc, các ô được tô đen là các ô được gán thành ô trống. Sau khi bước sinh vật cản hoàn tất, sinh các đối tượng còn lại vào các ô trống.

Để sinh vật cản, ban đầu toàn bộ ma trận được gán là các vật cản. Sau đó, thuật toán chọn ngẫu nhiên một ô làm điểm gốc (x, y) và bắt đầu đục các ô trống từ vị trí này. Với mỗi cụm, hệ thống sẽ tạo ra một trong hai dạng cấu trúc tia hoặc khung vuông. Trong đó, độ dài của tia và cạnh khung vuông được xác định ngẫu nhiên bằng weighted random. Nhờ đó, bản đồ hình thành các hành lang và khu vực khép kín, giúp bản đồ tự nhiên hơn thay vì phân bố ngẫu nhiên.

Sau khi tạo cấu trúc cơ bản, chương trình lần lượt đặt các thành phần quan trọng lên bản đồ, bao gồm ô đích (goal) của mỗi người chơi, các hộp (box) và vị trí xuất phát của người chơi cùng ô đích tương ứng. Các đối tượng này được phân bố ngẫu nhiên trên các ô hợp lệ (ô trống), sau đó được sao chép đối xứng để đảm bảo hai người chơi có điều kiện xuất phát tương đương. Cuối cùng, bản đồ được mã hóa lại thành các ký tự tương ứng và xuất ra tệp văn bản để sử dụng trong quá trình chạy trò chơi.

Độ phức tạp thuật toán. Giả sử bản đồ có kích thước $N \times N$. Quá trình sinh bản đồ chủ yếu bao gồm các thao tác duyệt ma trận, sinh vật cản và đối xứng hóa bản đồ. Mỗi bước chỉ cần

duyet qua số lượng hữu hạn các ô trong ma trận, do đó độ phức tạp thời gian tổng quát là $O(N^2)$. Trong khi đó, độ phức tạp bộ nhớ cũng là $O(N^2)$ do cần lưu trữ toàn bộ ma trận bản đồ và các cấu trúc phụ trợ liên quan.

3.2 Thuật toán AI

Các thuật toán như BFS, DFS và Dijkstra được lựa chọn làm nền tảng do kế thừa từ các lời giải đã được nghiên cứu và sử dụng thực tế bởi chính tác giả. Tuy nhiên, với việc thay đổi luật chơi so với Sokoban truyền thống (đặc biệt là cho phép đẩy nhiều hộp), không gian trạng thái trở nên phức tạp hơn đáng kể, đòi hỏi những phương pháp mạnh hơn để xử lý hiệu quả.

Không gian trạng thái của bài toán là cực kỳ lớn. Với bản đồ tiêu chuẩn có kích thước 16×16 , số trạng thái tối đa có thể đạt tới:

$$\left(\prod_{i=N-3}^N (N^2 - i) \right) \times 3^{(N^2-4)} \approx 7 \times 10^{129}$$

Trong đó, tích đầu tiên biểu diễn số cách sắp xếp của 4 đối tượng duy nhất (2 người chơi và 2 ô đích), còn lại các ô khác có thể nhận một trong ba trạng thái: ô trống, hộp hoặc vật cản. Với không gian lớn như vậy, các thuật toán như Minimax, MCTS hay Alpha-Beta pruning không thể hoạt động hiệu quả trong giới hạn thời gian 1 giây và thường dẫn đến các trạng thái lặp (cycle) hoặc bế tắc (deadlock).

Để giải quyết vấn đề trên, thuật toán Beam Search được lựa chọn nhờ khả năng giới hạn số lượng trạng thái mở rộng tại mỗi bước. Bằng cách chỉ giữ lại một số lượng nhỏ các trạng thái tốt nhất dựa trên heuristic, Beam Search giúp giảm đáng kể chi phí tính toán trong khi vẫn duy trì chất lượng lời giải ở mức chấp nhận được. Các tham số heuristic đã được tinh chỉnh (fine-tune) thông qua nhiều lần thử nghiệm nhằm đảm bảo hiệu quả ổn định trong môi trường thực tế.

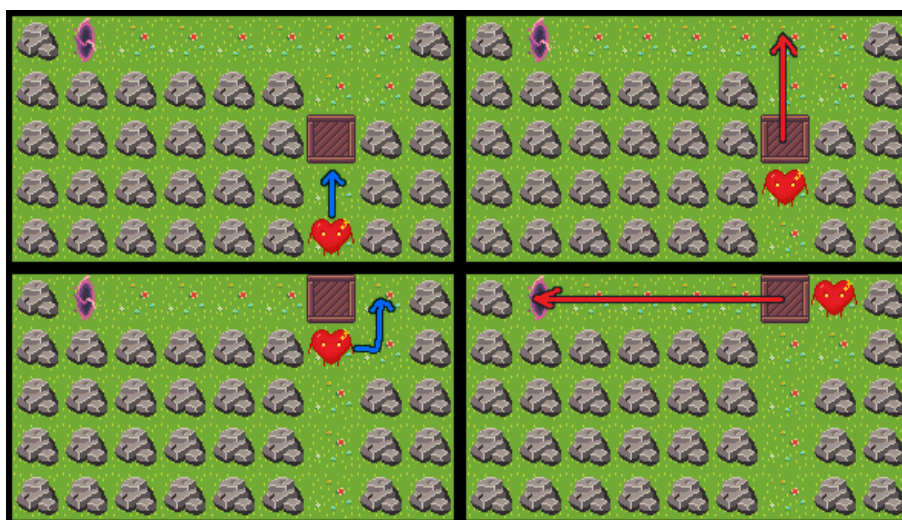
3.2.1 Thuật toán của tác giả

Phần này giới thiệu lại thuật toán đã được chính tác giả nghiên cứu, phát triển và sử dụng tại kỳ thi Tin học trẻ Toàn quốc Bảng C1 năm 2025. Một ý tưởng cốt lõi của giải thuật là đơn giản hóa bài toán nhiều hộp thành bài toán một hộp. Khi đó, quá trình giải bài toán một hộp bao gồm hai bước lặp đi lặp lại: (1) di chuyển nhân vật đến vị trí phù hợp cạnh hộp, và (2) đẩy hộp theo hướng mong muốn. Khi cần thay đổi hướng đẩy, nhân vật phải di chuyển sang vị trí khác của hộp trước khi tiếp tục. Chu trình này được lặp lại cho đến khi hộp được đưa tới ô đích.

Trong hai công đoạn trên, việc di chuyển được giải bằng BFS, đảm bảo tìm được đường đi ngắn nhất từ vị trí hiện tại đến vị trí mục tiêu. Trong khi đó, việc đẩy hộp được mô hình hóa thành một bài toán Dijkstra trên không gian trạng thái mở rộng kích thước $4 \times N \times N$, trong đó mỗi trạng thái bao gồm vị trí của hộp và hướng đẩy hiện tại. Việc sử dụng Dijkstra là cần thiết vì việc thay đổi hướng đẩy yêu cầu di chuyển vòng ra phía sau hộp, tạo ra chi phí khác nhau giữa các trạng thái.

Đối với bài toán nhiều hộp, tại mỗi thời điểm, hệ thống chỉ tập trung xử lý một hộp, trong khi các hộp còn lại được xem như vật cản. Việc lựa chọn hộp được thực hiện thông qua một hàm heuristic:

$$f(x) = w_1 \times \text{playerDist} + w_2 \times \text{goalDist} \quad (2)$$



Hình 2: Minh họa cơ chế hoạt động của thuật toán của tác giả. Trong đó, các mũi tên xanh mô tả công đoạn di chuyển đến cạnh hộp, mũi tên đỏ mô tả công đoạn đẩy hộp về phía trước.

Trong đó, $playerDist$ là khoảng cách Manhattan từ người chơi đến hộp, và $goalDist$ là khoảng cách Manhattan từ hộp đến ô đích. Các trọng số w_1, w_2 được tinh chỉnh thực nghiệm (hiện tại $w_1 = -2, w_2 = -1$) nhằm ưu tiên các hộp dễ tiếp cận và có tiềm năng ghi điểm cao, đồng thời hạn chế các trường hợp gây lặp trạng thái. Trọng số có thể âm, cho biết hệ số phạt của yếu tố được xét.

Do trò chơi hoạt động theo từng lượt với yêu cầu phản hồi nhanh, hệ thống AI không cần tìm toàn bộ chuỗi hành động hoàn chỉnh. Thay vào đó, một cơ chế dừng sớm được áp dụng: ngay khi tìm được một chuỗi hành động hợp lệ, thuật toán sẽ trả về bước đi đầu tiên. Cách tiếp cận này giúp giảm đáng kể thời gian tính toán trong khi vẫn đảm bảo chất lượng quyết định ở mức chấp nhận được.

Độ phức tạp thuật toán. Với bản đồ kích thước $N \times N$, mỗi lần BFS tìm đường cho người chơi có độ phức tạp $O(N^2)$ do BFS duyệt tối đa toàn bộ các ô trên bản đồ. Đối với bước đẩy hộp bằng Dijkstra trên không gian trạng thái mở rộng kích thước $4 \times N \times N$, duyệt qua các cạnh phải gọi lại thuật BFS, do đó độ phức tạp của Dijkstra là $O(N^4 \log N)$. Khi xét nhiều hộp, thuật toán cần đánh giá heuristic cho từng hộp và thử tìm lời giải tương ứng. Với B là số hộp, độ phức tạp tổng quát trong trường hợp xấu nhất là $O(B^2 \cdot N^4 \log N)$. Tuy nhiên, nhờ cơ chế dừng sớm và chỉ trả về hành động đầu tiên hợp lệ, độ phức tạp tổng quát giảm xuống còn $O(B \cdot N^4 \log N)$. Trong thực tế, thời gian chạy của thuật toán nhỏ hơn đáng kể so với thời gian lý thuyết. Bên cạnh đó, thuật toán có độ phức tạp bộ nhớ là $O(N^4)$ do đường đi của bước BFS được lưu bằng xâu ký tự.

3.2.2 Chiến lược chặn đường

Ngoài việc tập trung ghi điểm bằng cách đẩy hộp về ô đích, thuật toán còn được bổ sung một chiến lược mang tính đối kháng nhằm gây cản trở đối thủ khi đang dẫn điểm. Ý tưởng chính của chiến lược này là hạn chế khả năng tiếp cận hộp hoặc ô đích của đối phương, từ đó làm giảm tốc độ ghi điểm của đối thủ.

Thuật toán hoạt động theo nguyên tắc lựa chọn một trong 4 vị trí chiến thuật kề ô đích của đối thủ, sau đó di chuyển người chơi tới vị trí đó để chặn đường. Cụ thể, giải thuật xét bốn ô lân cận xung quanh ô đích của đối phương, sau đó đánh giá từng vị trí bằng một hàm heuristic dựa trên phần âm của tổng khoảng cách Manhattan từ vị trí chặn đến đối thủ và từ vị trí chặn đến hộp gần nhất.

Chiến lược này đặc biệt hiệu quả trên các bản đồ thông thoáng, nơi giải thuật có thể di chuyển nhanh giữa các ô lân cận của ô đích. Qua thực nghiệm và phân tích các trường hợp cụ

thể, nếu các ô kề với ô đích của đối thủ đều đủ thông thoáng để thuật toán có thể di chuyển qua lại linh hoạt, thì trong trường hợp thuật toán đang dẫn điểm và thành công tiếp cận khu vực này trước đối thủ, đối thủ không thể ghi điểm.

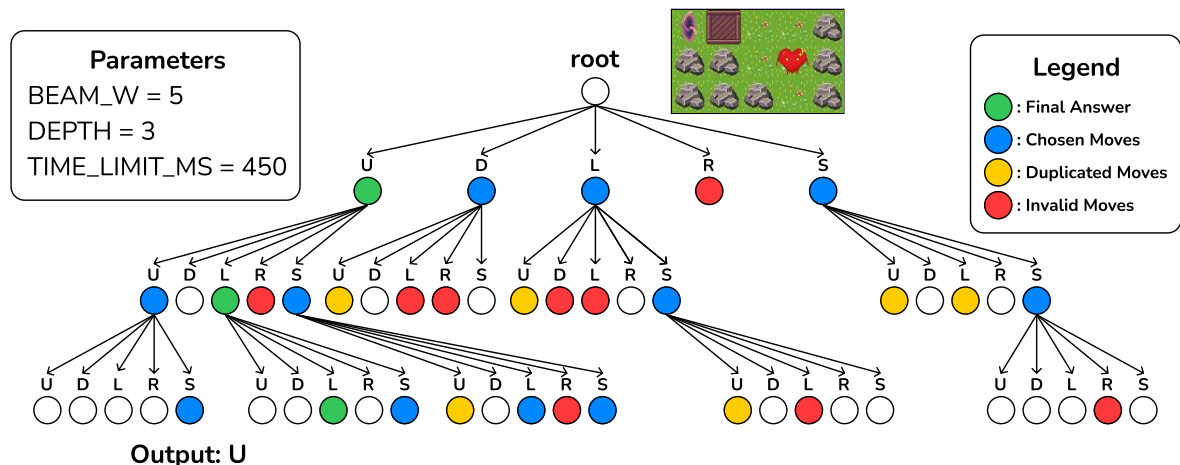
Nguyên nhân là do để đưa hộp vào ô đích, người chơi bắt buộc phải tiếp cận một trong các ô lân cận của ô đích nhằm thực hiện thao tác đẩy cuối cùng. Khi AI liên tục chiếm giữ và di chuyển luân phiên giữa các vị trí này, đối thủ sẽ không thể tiếp cận vị trí cần thiết để hoàn tất việc ghi điểm. Điều này biến chiến lược chặn đường trở thành một cơ chế phòng thủ hiệu quả trong giai đoạn cuối trận.

Độ phức tạp thuật toán. Thuật toán xét tối đa bốn vị trí lân cận quanh ô đích của đối thủ. Với mỗi vị trí, hệ thống thực hiện một lần BFS để kiểm tra khả năng tiếp cận. Do mỗi BFS có độ phức tạp $O(N^2)$, tổng độ phức tạp của chiến lược này vẫn là $O(N^2)$. Tương tự, thuật toán có độ phức tạp bộ nhớ là $O(N^2)$.

3.2.3 Thuật toán Beam Search

Để cải thiện chất lượng quyết định trong không gian trạng thái lớn và môi trường thời gian thực, thuật toán Beam Search [2025] được sử dụng cho chế độ khó của trò chơi. Đây là một biến thể của tìm kiếm theo chiều rộng (BFS), nhưng tại mỗi độ sâu chỉ giữ lại một số lượng hữu hạn các trạng thái tốt nhất (gọi là *beam width*), thay vì mở rộng toàn bộ không gian. Nhờ đó, thuật toán đạt được sự cân bằng giữa tốc độ và chất lượng lời giải.

Trong cài đặt của tác giả, Beam Search hoạt động với các tham số chính gồm độ rộng beam $BEAM_W = 90$, độ sâu tìm kiếm $DEPTH = 16$, và giới hạn thời gian ≈ 450 ms. Thuật toán bắt đầu từ trạng thái gốc (root), sau đó lặp theo từng tầng (depth). Tại mỗi bước, tất cả các hành động hợp lệ từ các trạng thái hiện tại được sinh ra thông qua hàm `applyMove`. Các trạng thái mới sau đó được đánh giá bằng một hàm heuristic và sắp xếp theo giá trị giảm dần. Chỉ $BEAM_W$ trạng thái tốt nhất được giữ lại cho bước tiếp theo.



Hình 3: Minh họa cơ chế hoạt động của thuật toán Beam Search trên cây trạng thái trò chơi (State tree). Thuật toán duyệt đến độ sâu 3 nước tiếp theo, trong đó mỗi tầng chỉ giữ lại tối đa 5 đỉnh. Các đỉnh màu xanh lá, xanh dương, vàng, đỏ lần lượt là các đỉnh được đánh giá là tốt nhất và được chọn cuối cùng, các đỉnh được giữ lại ở mỗi tầng, các đỉnh có trạng thái bị trùng lặp, các nước đi không hợp lệ.

Một điểm quan trọng trong việc cài đặt là sử dụng Zobrist Hashing để mã hóa trạng thái. Mỗi cấu hình của trò chơi (bao gồm vị trí người chơi, hộp và điểm số) được ánh xạ thành một giá trị hash 64-bit, cho phép phát hiện và loại bỏ nhanh các trạng thái trùng lặp. Điều này giúp giảm đáng kể số lượng trạng thái cần xét trong không gian tìm kiếm. Nhờ đó, hiệu năng của mô

hình được cải thiện rõ rệt.

Hàm heuristic đóng vai trò quyết định trong việc đánh giá chất lượng trạng thái. Do đó, hàm heuristic trong thuật toán này đòi hỏi phải bao quát được nhiều yếu tố hơn so với hai chế độ dễ và thường. Giá trị heuristic được xây dựng dựa trên nhiều yếu tố: chênh lệch điểm số, tổng khoảng cách từ các hộp đến ô đích của người chơi và đối thủ, mức độ “di động” (mobility) của các hộp, khoảng cách từ người chơi đến hộp gần nhất, cũng như các yếu tố chiến thuật như khả năng chặn ô đích của đối phương. Ngoài ra, heuristic còn tích hợp các cơ chế phạt để tránh dao động qua lại (oscillation) hoặc đứng yên không cần thiết, giúp cải thiện tính ổn định của lời giải và tránh rơi vào cycle.

Trong phạm vi đề án này, ta giả sử có k yếu tố được xem như ảnh hưởng đến một trạng thái, hàm heuristic có thể được xem như một ánh xạ tuyến tính:

$$h(x) = \sum_i w_i x_i = w_1 x_1 + w_2 x_2 + \dots + w_k x_k \quad (3)$$

Trong đó, x_i là giá trị của yếu tố thứ i (như chênh lệch điểm số hay khoảng cách), w_i là trọng số của yếu tố thứ i . Tương tự như thuật toán của tác giả, trọng số của một yếu tố cũng có thể âm, cho biết yếu tố đó có giá trị càng lớn thì trạng thái càng xấu. Giá trị w_i là hệ số phạt của yếu tố đó. Các giá trị này được tinh chỉnh sao cho thuật toán tối ưu. Ví dụ, chênh lệch điểm số là ưu tiên hàng đầu, yếu tố này có giá trị lớn, hay việc đi ra xa hộp làm tăng khoảng cách được xem như không tốt, nên yếu tố này có trọng số là một số âm.

Do ràng buộc thời gian thực, thuật toán không cần tìm toàn bộ lời giải hoàn chỉnh. Thay vào đó, mỗi trạng thái lưu trữ chuỗi hành động đầu tiên (firstPlan), và sau khi kết thúc tìm kiếm (hoặc hết thời gian), thuật toán sẽ trả về hành động đầu tiên của trạng thái tốt nhất tìm được. Cách tiếp cận này phù hợp với mô hình chơi theo lượt, đồng thời đảm bảo thời gian phản hồi nhanh.

Độ phức tạp thuật toán. Khác với BFS truyền thống có độ phức tạp lũy thừa theo độ sâu, Beam Search chỉ giữ lại tối đa BEAM_W trạng thái tại mỗi tầng. Do đó, số trạng thái được mở rộng tối đa là $O(\text{BEAM_W} \times \text{DEPTH} \times K)$. Trong đó, K là giới hạn để tính toán hàm đánh giá heuristic. Trong đánh giá của đề án, $K \approx N^2$.

3.2.4 Các chế độ AI

Hệ thống AI được chia thành ba mức độ:

- **Chế độ dễ:** Áp dụng thuật toán cốt lõi (chi tiết về thuật toán, tham khảo mục 3.2.1), xem đối thủ và ô đích của đối thủ như vật cản. Tuy nhiên, để giảm độ khó của thuật toán, áp dụng cơ chế di chuyển ngẫu nhiên với tỉ lệ 20% và loại bỏ cơ chế chọn hộp Heuristic, hộp được chọn cố định là hộp cuối cùng trong danh sách hộp.
- **Chế độ thường:** Mở rộng từ chế độ dễ, bổ sung chiến lược chặn đường đến ô đích của đối thủ khi đang dẫn điểm (chi tiết về thuật toán, tham khảo mục 3.2.2).
- **Chế độ khó:** Sử dụng thuật toán Beam Search. Chi tiết hơn về thuật toán, tham khảo mục 3.2.3.

4 Thực nghiệm

4.1 Quá trình thực hiện

4.1.1 Tuần 1

Trong tuần đầu tiên, trọng tâm của đề án là xây dựng nền tảng ban đầu cho trò chơi. Tác giả tiến hành cụ thể hóa ý tưởng đã có từ trước, đồng thời lập kế hoạch chi tiết cho toàn bộ quá trình phát triển (chi tiết hơn về bảng kế hoạch, tham khảo mục 4.2). Song song đó, các đối tượng trong trò chơi được thiết kế thủ công bằng phần mềm vẽ pixel art Aseprite. Sườn của trò chơi cũng được triển khai, bao gồm điều khiển hai nhân vật, xử lý logic trò chơi, sinh bản đồ và hiển thị giao diện bằng thư viện đồ họa. Kết thúc tuần này, trò chơi đã có thể vận hành với giao diện cơ bản, tuy nhiên các tính năng nâng cao như đấu với máy hoặc giao diện thiết lập vẫn chưa được hoàn thiện và chủ yếu phải cấu hình thủ công qua mã nguồn.

4.1.2 Tuần 2

Ở tuần thứ hai, trò chơi được mở rộng với việc bổ sung các giao diện tương tác, bao gồm màn hình chính và màn hình thiết lập trò chơi và màn hình trò chơi. Điều này giúp cải thiện đáng kể trải nghiệm người dùng khi không còn phụ thuộc vào việc chỉnh sửa trực tiếp mã nguồn. Đồng thời, tác giả bắt đầu xây dựng cơ chế tương tác cho các bot nhằm chuẩn bị cho chế độ chơi với máy. Tuy nhiên, tại thời điểm này, các bot vẫn hoạt động theo cơ chế ngẫu nhiên hoàn toàn, do đó chỉ mang tính chất minh họa và chưa thể hiện được tính chiến thuật.

4.1.3 Tuần 3

Tuần thứ ba tập trung vào việc nâng cao chất lượng trò chơi và bổ sung tính năng AI. Một số tính năng hỗ trợ người chơi được bổ sung nhằm cải thiện trải nghiệm tổng thể. Quan trọng hơn, tác giả tiến hành nghiên cứu, thử nghiệm và lựa chọn các thuật toán phù hợp để xây dựng hệ thống AI. Các giải thuật được hiện thực hóa và tích hợp vào trò chơi, từ đó hình thành ba mức độ AI: dễ, thường và khó. Thuật toán Beam Search cũng được triển khai ở mức thử nghiệm, tuy nhiên vẫn đang trong quá trình đánh giá hiệu năng trước khi tích hợp chính thức vào hệ thống. Mọi chi tiết hơn về giải thuật, tham khảo mục 3.2.

4.1.4 Tuần 4

Trong tuần cuối, mục tiêu chính là hoàn thiện hệ thống, bao gồm chạy thử kết quả, tối ưu hiệu năng, tinh chỉnh từng mức độ khó của AI và hoàn tất báo cáo. Toàn bộ các màn hình được tái cấu trúc. Các hoạt động như thử nghiệm với người dùng, thu thập phản hồi và tinh chỉnh trò chơi cũng được tiến hành nhằm đảm bảo tính ổn định và chất lượng sản phẩm. Đồng thời, việc hoàn thiện tài liệu, chạy demo và tổng hợp kết quả cũng được thực hiện. Chi tiết hơn về kết quả thực nghiệm và demo, tham khảo mục 4.3 và A.

4.2 Bảng tự phân công

Kết quả thực hiện đề án được tóm tắt trong Bảng 1. Ngoài ra, về mã nguồn và kết quả chi tiết của đề án có thể được tham khảo tại github.com/lftroq/IT003.Q21.CTTN-Project-2P-Sokoban, cũng như trong phần phụ lục của đề án.

Bảng 1: Bảng tự phân công thực hiện đồ án

Công việc	Chi tiết	Thời gian	Tiến độ
Lên kế hoạch	Lên bảng kế hoạch cụ thể cho đồ án môn học.	Tuần 1	Hoàn thành
Phân tích tính năng	Giao diện đồ họa, các tính năng, các chế độ chơi, điều khiển, ...	Tuần 1	Hoàn thành
Thiết kế giao diện	Vẽ các đối tượng trong trò chơi, phác thảo giao diện trò chơi. Code triển khai đồ họa.	Tuần 1–2	Hoàn thành
Thiết kế khung sườn	Xây dựng cấu trúc chính của trò chơi	Tuần 1–2	Hoàn thành
Mở rộng chức năng	Thêm menu, tiện ích và màn hình phụ	Tuần 2–3	Hoàn thành
Tính năng nâng cao	Chế độ người vs người, người vs máy, máy vs máy (mô phỏng). Máy có nhiều độ khó khác nhau, thử nghiệm các thuật toán khác nhau.	Tuần 3	Hoàn thành
Kiểm thử, tối ưu	Thu thập phản hồi, cải thiện hiệu năng	Tuần 3–4	Hoàn thành
Chạy thử kết quả	Cho các máy đấu với nhau ở từng mức độ khó, tinh chỉnh các tham số cho phù hợp	Tuần 4	Hoàn thành
Hoàn thiện	Hoàn thiện đồ án, triển khai và viết báo cáo	Tuần 4	Hoàn thành

4.3 Phân tích kết quả thực nghiệm

Dựa trên kết quả thực nghiệm thu được từ quá trình cho các bot thi đấu tự động trên nhiều cấu hình bản đồ và giới hạn bước đi khác nhau (xem chi tiết tại mục A.2), có thể rút ra một số nhận xét đáng chú ý về chất lượng và hành vi của các thuật toán AI đã được xây dựng.

Quy ước mỗi cấu hình thử nghiệm được xem như một *set*, trong đó hai bot thi đấu tổng cộng 20 trận. Một bot được xem là thắng *set* nếu số trận thắng của bot đó lớn hơn đối thủ trong cùng cấu hình.

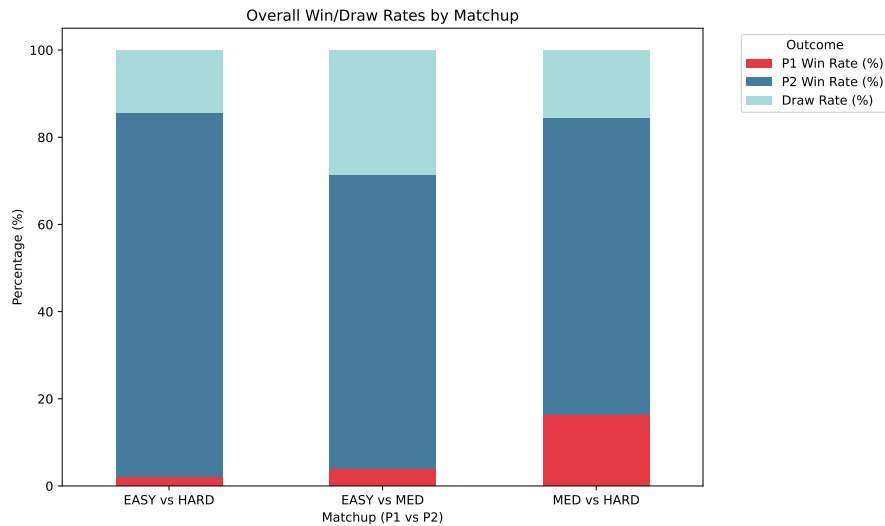
4.3.1 Đánh giá tổng quan năng lực

Bảng 2: Tổng hợp kết quả thi đấu giữa các bot (Tổng số: 720 trận / 36 set mỗi cặp đấu)

Cặp đấu (P1 vs P2)	Trận / Set P1 thắng	Trận / Set P2 thắng	Trận / Set hòa
EASY vs MED	29 (4.0%) / 1 (2.8%)	486 (67.5%) / 35 (97.2%)	205 (28.5%) / 0 (0%)
EASY vs HARD	16 (2.2%) / 0 (0%)	600 (83.3%) / 36 (100%)	104 (14.5%) / 0 (0%)
MED vs HARD	117 (16.3%) / 0 (0%)	492 (68.3%) / 36 (100%)	111 (15.4%) / 0 (0%)

Dựa vào biểu đồ 4 và bảng 2, có thể thấy kết quả đối đầu tổng thể phản ánh sự phân hóa trình độ rõ rệt giữa ba cấp độ bot, đúng với thiết kế ban đầu của trò chơi:

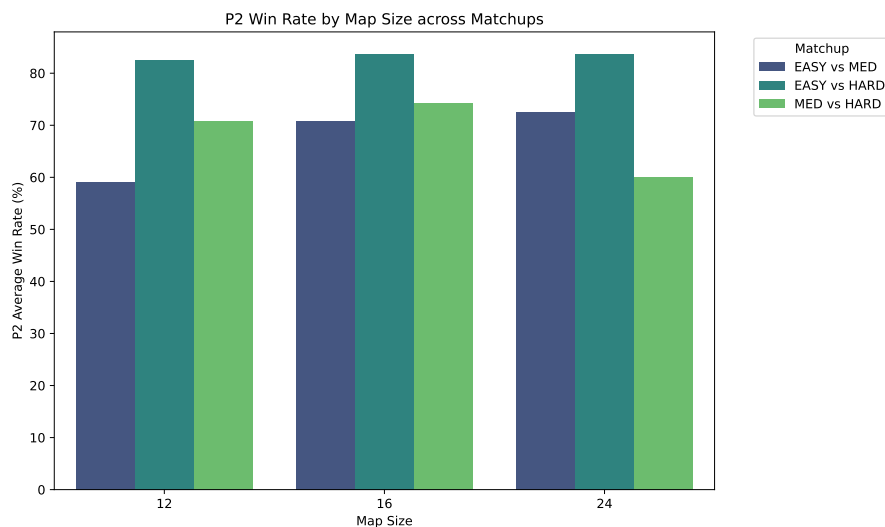
- **EASY vs MED:** Bot MED áp đảo hoàn toàn với tỉ lệ thắng lên tới 67.5% (486/720 trận) và thắng 35/36 set. Sự kết hợp giữa Heuristic chọn hộp và cơ chế tấn công giúp MED vượt trội hơn hẳn so với EASY (chỉ thắng 4.0%). Tỉ lệ hòa ở cặp đấu này cũng chiếm phần khá lớn với 28.5%. Điều này có thể giải thích được vì thuật toán của 2 bot này được xây dựng trên cùng một thuật toán cơ sở.



Hình 4: Tỷ lệ thắng từng cặp trận của các bot khi so với nhau. Trong đó, Phần màu đỏ là tỷ lệ thắng của P1 (bot yếu hơn), phần màu xanh đậm là tỷ lệ thắng của P2 (bot mạnh hơn), phần màu xanh nhạt là tỷ lệ hòa.

- EASY vs HARD: Sự chênh lệch đạt mức tuyệt đối khi HARD thắng toàn bộ 36/36 set (chiếm 83.3% tổng số trận). Điều này chứng minh khoảng cách từ một thuật toán cơ sở (EASY) đến thuật toán Heuristic nâng cao (HARD) là rất lớn.
- MED vs HARD: Dù MED có khả năng cạnh tranh tốt hơn (thắng 16.3% số trận), HARD vẫn chứng minh tính ổn định tuyệt đối bằng việc giành chiến thắng 36/36 set (68.3% số trận). Khả năng kiểm soát nhiều hộp cùng lúc và hàm đánh giá phức tạp giúp HARD luôn duy trì được ưu thế.

4.3.2 Ảnh hưởng của kích thước bản đồ



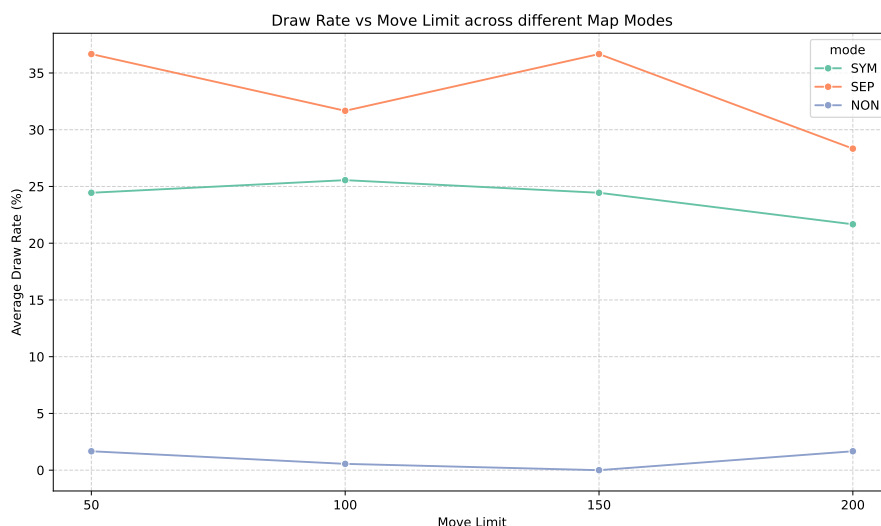
Hình 5: Tỷ lệ thắng của các bot mạnh theo từng cặp đấu và từng kích thước map. Trong đó, cột màu xanh đậm thể hiện cặp đấu EASY vs MED, cột màu xanh lơ thể hiện cặp đấu EASY vs HARD, cột màu xanh lá thể hiện cặp đấu MED vs HARD.

Dựa vào hình 5, có thể thấy kích thước không gian có tác động đáng kể đến khả năng triển khai chiến thuật của các thuật toán. Trong tất cả các cặp đấu, bot thứ hai (P2 - bot cấp cao hơn) luôn duy trì tỷ lệ thắng áp đảo trên 60% ở mọi kích thước bản đồ (Size 12, 16, và 24).

Đáng chú ý, ở kích thước bản đồ tầm trung (Size 16), tỷ lệ thắng của bot HARD khi đối đầu

với MED và EASY đều đạt đỉnh cao nhất. Điều này cho thấy với không gian mở vừa đủ, bot HARD có thể tối ưu hóa thuật toán tìm đường và kiểm soát hộp tốt nhất trước khi bị cản trở bởi quãng đường vượt quá giới hạn độ sâu tìm kiếm (như ở map 24) hoặc bị ép góc quá sớm (như ở map 12).

4.3.3 Tác động của chế độ bản đồ và giới hạn số nước đi



Hình 6: Tác động của chế độ bản đồ và giới hạn số nước đi. Trong đó, đường màu cam thể hiện chế độ map tách rời nhau, đường màu xanh lá thể hiện chế độ map tiêu chuẩn, đường màu xanh dương thể hiện chế độ map không vật cản.

Trò chơi ghi nhận một lượng lớn các trận hòa, và hình 6 cho thấy nguyên nhân chính xuất phát từ thiết kế cấu trúc bản đồ chứ không phải do giới hạn bước đi. Sự chênh lệch giữa các chế độ cho thấy chế độ SEP (Separated) luôn duy trì tỉ lệ hòa rất cao (dao động từ 30% - 35%) bất kể giới hạn bước đi là 50 hay 200. Ngược lại, chế độ NON (Không vật cản) có tỉ lệ hòa cực thấp, gần như bằng 0%.

Nguyên nhân thực tiễn của hiện tượng này đến từ việc chia cắt khu vực (SEP) hoặc tạo bản đồ đối xứng hoàn toàn (SYM) để tạo ra các thế bế tắc (deadlock). Ngoài ra, bộ sinh bản đồ ngẫu nhiên đôi khi tiềm ẩn nguy cơ tạo ra các vách ngăn khiến bot không thể tiếp cận hộp hoặc đích đến, làm cho trận đấu bị đóng băng và số bước đi cạn kiệt. Việc tăng giới hạn bước đi dù có tác động tích cực đến tỉ lệ hòa, song không giải quyết triệt để được vấn đề này.

4.3.4 Kết luận

Mức độ khó của các bot đã được phân hóa rất tốt. HARD là bot có hiệu năng toàn diện nhất nhờ mô hình thuật toán tân tiến, cho phép xử lý không gian đa mục tiêu hiệu quả. MED có sự cải tiến rõ rệt so với EASY nhưng vẫn bộc lộ hạn chế khi đối đầu với chiến thuật kiểm soát phức tạp của HARD. Để tối ưu hóa trải nghiệm trò chơi trong tương lai, bên cạnh việc tinh chỉnh AI, bộ sinh bản đồ cần được cải tiến thêm (đặc biệt đối với các chế độ có vách ngăn như SEP) nhằm hạn chế tối đa các góc chết không thể giải quyết, từ đó giảm tỉ lệ hòa không mong muốn.

5 Tổng kết

5.1 Kết quả đạt được

Trong phạm vi đồ án này, tác giả đã xây dựng thành công một ứng dụng trò chơi đẩy hộp kết hợp yếu tố đối kháng, dựa trên nền tảng của Sokoban truyền thống. Không chỉ dừng lại ở việc tái hiện lại cơ chế gameplay quen thuộc, trò chơi đã được mở rộng đáng kể với các luật chơi mới như tương tác đồng thời giữa hai người chơi, khả năng đẩy nhiều hộp, cũng như yếu tố cạnh tranh thông qua hệ thống tính điểm. Điều này đồng thời đã biến bài toán xây dựng lời giải từ một bài toán giải đồ thị thành một môi trường động có tính chiến thuật cao.

Về mặt kỹ thuật, đồ án đã triển khai đầy đủ các thành phần quan trọng của một ứng dụng trò chơi hoàn chỉnh, bao gồm sinh bản đồ ngẫu nhiên có kiểm soát, giao diện đồ họa trực quan, hệ thống điều khiển người chơi, cũng như các chế độ chơi đa dạng. Đặc biệt, phần AI đã được nghiên cứu, thực nghiệm và xây dựng với nhiều mức độ phức tạp khác nhau, từ hành vi ngẫu nhiên đến các chiến lược dựa trên thuật toán tìm kiếm và heuristic. Việc kết hợp các thuật toán như BFS, Dijkstra cùng với Beam Search cho thấy một hướng tiếp cận hiệu quả trong việc xử lý bài toán có không gian trạng thái lớn và yêu cầu thời gian phản hồi nhanh.

5.2 Hạn chế

Bên cạnh những kết quả đạt được, đồ án vẫn còn tồn tại một số hạn chế cần được khắc phục trong các phiên bản tiếp theo.

Thứ nhất, quá trình sinh bản đồ ngẫu nhiên hiện tại chưa đảm bảo tính khả thi hoàn toàn. Cụ thể, các đối tượng quan trọng như hộp hoặc cổng dịch chuyển có thể bị bao quanh hoàn toàn bởi vật cản, dẫn đến việc người chơi không thể tiếp cận hoặc tương tác. Điều này ảnh hưởng trực tiếp đến tính công bằng và trải nghiệm của trò chơi.

Thứ hai, hệ thống chưa có cơ chế kiểm soát thời gian thực thi chặt chẽ đối với các bot. Trong trường hợp một bot rơi vào vòng lặp vô hạn hoặc mất quá nhiều thời gian để tính toán, toàn bộ trò chơi có thể bị treo do không có cơ chế timeout để ngắt tiến trình.

Thứ ba, các thuật toán của chế độ máy sử dụng cơ chế đánh giá heuristic vẫn chưa có phương pháp tinh chỉnh phù hợp, dẫn đến chất lượng ra quyết định chưa thật sự ổn định trong nhiều tình huống phức tạp. Các trọng số heuristic hiện chủ yếu được lựa chọn thủ công dựa trên kinh nghiệm, cảm quan và nhận xét của chính tác giả. Do đó, bot đôi khi đưa ra những hành động chưa tối ưu hoặc thiếu tính thích nghi trước các trạng thái bản đồ khác nhau.

Ngoài ra, trò chơi hiện vẫn chưa được tích hợp hệ thống âm thanh và nhạc nền phù hợp. Điều này khiến trải nghiệm chơi chưa thật sự sinh động và thiếu các phản hồi âm thanh cần thiết trong quá trình tương tác. Các hiệu ứng như di chuyển, đẩy hộp, ghi điểm hoặc kết thúc trận đấu hiện chỉ được thể hiện thông qua hình ảnh, làm giảm mức độ hấp dẫn và cảm giác phản hồi của trò chơi đối với người chơi.

Những hạn chế trên là cơ sở quan trọng để định hướng cho các cải tiến trong tương lai, đặc biệt là về độ ổn định, tính mở rộng và khả năng triển khai thực tế của hệ thống.

5.3 Hướng phát triển trong tương lai

Trong các phiên bản tiếp theo, trò chơi sẽ được cải tiến theo nhiều hướng nhằm nâng cao độ ổn định, tính mở rộng và chất lượng trải nghiệm người chơi.

Trước hết, cần khắc phục các hạn chế hiện tại của đồ án. Cụ thể, quá trình sinh bản đồ cần



được bổ sung các bước kiểm tra tính khả thi (reachability) để đảm bảo các đối tượng quan trọng như hộp và cổng dịch chuyển luôn có thể tiếp cận được. Đồng thời, hệ thống cần tích hợp cơ chế giới hạn thời gian thực thi (timeout) cho các bot nhằm tránh tình trạng treo chương trình.

Bên cạnh đó, trải nghiệm người chơi có thể được nâng cao thông qua việc bổ sung các cơ chế như xem trước bản đồ (map preview) ngay tại màn hình khởi tạo. Điều này giúp người chơi có thể đánh giá nhanh mức độ phù hợp của bản đồ trước khi bắt đầu trận đấu. Một hướng tiếp cận khác là cải tiến hoàn toàn thuật toán sinh map để đảm bảo chất lượng bản đồ ngay từ đầu, giảm sự phụ thuộc vào yếu tố ngẫu nhiên.

Về trải nghiệm người chơi, có thể phát triển bằng cách bổ sung thêm phần âm nhạc, giúp trò chơi sinh động hơn. Bên cạnh đó, xây dựng chế độ chơi trực tuyến (online multiplayer) cho phép người chơi kết nối và thi đấu với nhau thông qua mạng, từ đó mở rộng đáng kể tính cạnh tranh và khả năng ứng dụng thực tế của trò chơi.

Cuối cùng, hệ thống AI vẫn còn nhiều tiềm năng để cải thiện. Trước mắt, việc cải tiến heuristic bằng cách tìm một phương pháp để đánh giá các trạng thái một cách phù hợp và mở rộng khả năng đánh giá trạng thái cũng sẽ góp phần giúp AI đưa ra các quyết định chính xác và ổn định hơn trong môi trường có không gian trạng thái lớn. Tuy nhiên, mô hình mạnh hơn đồng nghĩa với việc đánh đổi thời gian xử lý, việc nghiên cứu thêm các hướng tối ưu hóa tốc độ là cần thiết để có thể triển khai các phương pháp mạnh mẽ hơn.



Tài liệu

Hệ thống thi Tin học trẻ Toàn quốc. Hướng dẫn nội dung chi tiết bảng c1 vòng chung kết năm 2025, 2025. URL <https://tinhoctre.vn/post/108-huongdan-noidung-bangc1-vck2025>. Truy cập ngày: 04/05/2026.

Hiroyuki Imabayashi. *Sokoban*. Thinking Rabbit, Japan, 1982.

Sergey Pastukhov. Solving sokoban using hierarchical reinforcement learning with landmarks. *arXiv preprint arXiv:2504.04366*, 2025. doi: 10.48550/arXiv.2504.04366. URL <https://arxiv.org/abs/2504.04366>.

Pete Shinnars. Pygame news, 2000. URL <https://www.pygame.org/news>. Truy cập ngày: 04/05/2026.

Wikipedia contributors. Sokoban — wikipedia, the free encyclopedia, 2024. URL <https://en.wikipedia.org/wiki/Sokoban>. Truy cập ngày: 04/05/2026.

Wikipedia contributors. Beam search — wikipedia, the free encyclopedia, 2025. URL https://en.wikipedia.org/wiki/Beam_search. Truy cập ngày: 10/05/2026.

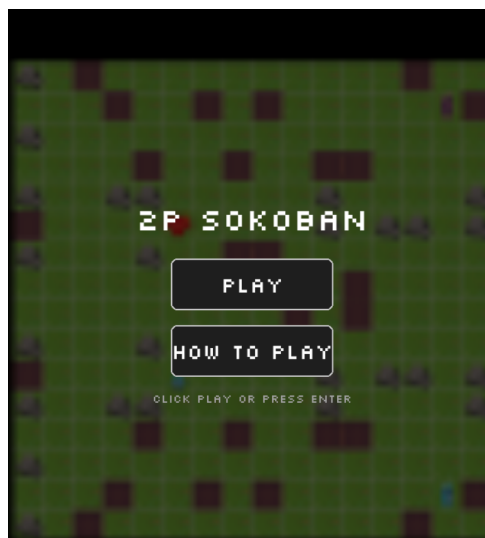
A Phụ lục

A.1 Giới thiệu (demo) kết quả

Kết quả demo có thể được tham khảo tại youtube.com/shorts/8atOSKX8_Sk.

A.1.1 Màn hình chính

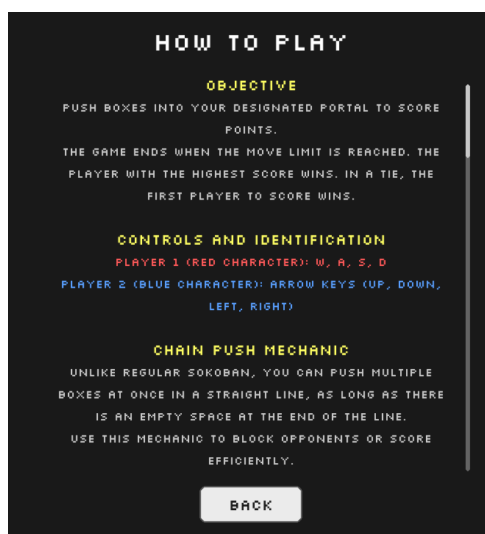
Màn hình chính là giao diện đầu tiên xuất hiện khi khởi động trò chơi. Tại đây, người chơi có thể lựa chọn các bắt đầu chơi để di chuyển đến giao diện thiết lập trò chơi hoặc truy cập phần hướng dẫn hoặc thoát trò chơi.



Hình 7: Màn hình chính của trò chơi, gồm có 2 nút để bắt đầu chơi hoặc vào phần hướng dẫn.

A.1.2 Màn hình hướng dẫn

Màn hình hướng dẫn cung cấp thông tin cơ bản về luật chơi, cơ chế điều khiển và các đối tượng xuất hiện trong trò chơi. Điều này giúp người chơi có thông tin về các cơ chế đặc biệt như đẩy nhiều hộp, cổng dịch chuyển và hệ thống tính điểm.



Hình 8: Màn hình hướng dẫn của trò chơi, người chơi có thể xem hướng dẫn bằng cách sử dụng con lăn chuột hoặc chọn quay về màn hình chính.

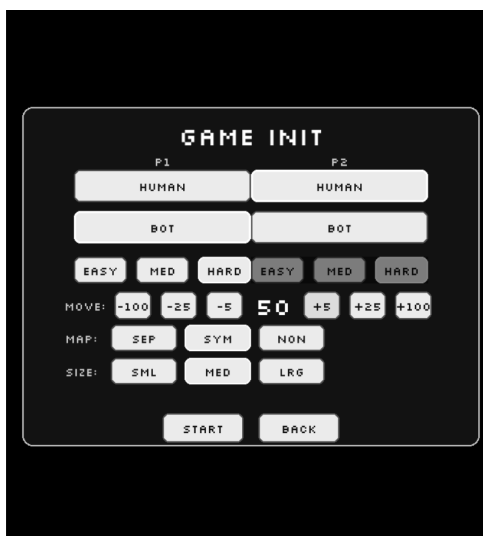
A.1.3 Màn hình thiết lập

Màn hình thiết lập cho phép người chơi tùy chỉnh các cấu hình trước khi bắt đầu trận đấu. Tại đây, người chơi có thể lựa chọn chế độ điều khiển cho từng người chơi (người hoặc bot AI), mức độ khó của bot, giới hạn số bước đi, kiểu bản đồ và kích thước bản đồ.

Các chế độ bản đồ bao gồm:

- SEP: Hai người chơi thi đấu trên hai khu vực tách biệt.
- SYM: Bản đồ đối xứng tiêu chuẩn với các vật cản.
- NON: Bản đồ tự do không có vật cản.

Ngoài ra, người chơi cũng có thể lựa chọn nhiều kích thước bản đồ khác nhau để thay đổi độ phức tạp của trận đấu và trải nghiệm chơi. Người chơi có thể quay về màn hình chính hoặc bắt đầu trò chơi sau khi thiết lập hoàn tất.



Hình 9: Màn hình thiết lập của trò chơi, gồm có các nút giúp thiết lập các cấu hình của trò chơi và các nút để di chuyển về màn hình chính hoặc bắt đầu vào trò chơi.

A.1.4 Màn hình trò chơi

Màn hình trò chơi là nơi diễn ra toàn bộ quá trình thi đấu giữa hai người chơi hoặc giữa người chơi với bot hoặc giữa hai bot. Tại đây, hệ thống hiển thị bản đồ, vị trí người chơi, hộp, vật cản, cổng dịch chuyển, điểm số và số bước còn lại của trận đấu. Các cơ chế tương tác đồng thời và đẩy hộp được xử lý theo thời gian thực nhằm đảm bảo trải nghiệm chơi ổn định.

Ngoài khu vực gameplay chính, trò chơi còn hỗ trợ một số nút chức năng nhằm tăng tính tiện lợi trong quá trình chơi:

- UNDO: Hoàn tác trạng thái của một lượt trước đó của trò chơi.
- RESET: Khởi động lại trận đấu với cùng bản đồ hiện tại. Hay nói cách khác, hoàn tác trạng thái trở về lượt đầu tiên của trò chơi
- NEW: Trở về màn hình thiết lập để bắt đầu một trận đấu mới
- END: Kết thúc trận đấu hiện tại. Tính điểm dựa trên kết quả từ lượt hiện tại.
- QUIT: Trở về màn hình chính của trò chơi.



Hình 10: Màn hình trò chơi, gồm có bản đồ chính, các thông tin của trò chơi như điểm số, số lượt còn lại, trạng thái của bot (chế độ người vs bot), các nút chức năng hỗ trợ.

A.2 Kết quả thực nghiệm

A.2.1 Thiết lập thực nghiệm

Phần này trình bày chi tiết kết quả thực nghiệm của hệ thống AI thông qua quá trình chạy thử nghiệm có hệ thống (sweep result) trên nhiều cấu hình khác nhau. Tổng cộng có:

- 3 kích thước bản đồ: 12, 16, 24.
- 3 chế độ bản đồ: SYM, SEP, NON.
- 4 mức giới hạn số bước đi: 50, 100, 150, 200.
- 3 cặp đấu: EASY vs MED, EASY vs HARD, MED vs HARD.

Trong đó, mỗi cấu hình được chạy 20 trận.

Trong cài đặt của tác giả, các bot đều có giới hạn thời gian tối đa (time limit) là 450 ms. Giả sử xét giới hạn số bước đi trung bình là 125 bước, thời gian tối đa ước tính để chạy thí nghiệm là:

$$\frac{3 \times 3 \times 125 \times 3 \times 20 \times 450}{1000} = 30375(s) \approx 8.4(h)$$

Tuy nhiên, trong thực tế, không phải lúc nào cả hai bot cũng hoạt động hết giới hạn thời gian cho phép, thời gian chạy thí nghiệm thực tế chỉ khoảng gần 6 giờ.

A.2.2 Bảng kết quả chi tiết

Dưới đây là toàn bộ kết quả chi tiết của kết quả thực nghiệm.

Bảng 3: Bảng kết quả chi tiết thực nghiệm

Size	Mode	Limit	Runs	P1	P2	P1 Wins	P2 Wins	Draw
12	SYM	50	20	EASY	MED	0	12	8
12	SYM	50	20	EASY	HARD	0	18	2
12	SYM	50	20	MED	HARD	3	14	3
12	SYM	100	20	EASY	MED	0	13	7
12	SYM	100	20	EASY	HARD	1	17	2
12	SYM	100	20	MED	HARD	4	10	6
12	SYM	150	20	EASY	MED	2	8	10



Size	Mode	Limit	Runs	P1	P2	P1 Win(s)	P2 Win(s)	Draw(s)
12	SYM	150	20	EASY	HARD	0	14	6
12	SYM	150	20	MED	HARD	3	15	2
12	SYM	200	20	EASY	MED	5	10	5
12	SYM	200	20	EASY	HARD	0	16	4
12	SYM	200	20	MED	HARD	3	15	2
12	SEP	50	20	EASY	MED	0	5	15
12	SEP	50	20	EASY	HARD	1	15	4
12	SEP	50	20	MED	HARD	2	15	3
12	SEP	100	20	EASY	MED	2	6	12
12	SEP	100	20	EASY	HARD	1	12	7
12	SEP	100	20	MED	HARD	0	16	4
12	SEP	150	20	EASY	MED	4	2	14
12	SEP	150	20	EASY	HARD	4	12	4
12	SEP	150	20	MED	HARD	4	6	10
12	SEP	200	20	EASY	MED	5	7	8
12	SEP	200	20	EASY	HARD	1	14	5
12	SEP	200	20	MED	HARD	2	13	5
12	NON	50	20	EASY	MED	0	19	1
12	NON	50	20	EASY	HARD	0	20	0
12	NON	50	20	MED	HARD	6	14	0
12	NON	100	20	EASY	MED	0	20	0
12	NON	100	20	EASY	HARD	0	20	0
12	NON	100	20	MED	HARD	2	18	0
12	NON	150	20	EASY	MED	0	20	0
12	NON	150	20	EASY	HARD	0	20	0
12	NON	150	20	MED	HARD	2	18	0
12	NON	200	20	EASY	MED	0	20	0
12	NON	200	20	EASY	HARD	0	20	0
12	NON	200	20	MED	HARD	3	16	1
16	SYM	50	20	EASY	MED	0	11	9
16	SYM	50	20	EASY	HARD	0	18	2
16	SYM	50	20	MED	HARD	1	15	4
16	SYM	100	20	EASY	MED	0	17	3
16	SYM	100	20	EASY	HARD	0	17	3
16	SYM	100	20	MED	HARD	3	11	6
16	SYM	150	20	EASY	MED	1	11	8
16	SYM	150	20	EASY	HARD	0	15	5
16	SYM	150	20	MED	HARD	4	12	4
16	SYM	200	20	EASY	MED	0	13	7
16	SYM	200	20	EASY	HARD	1	16	3
16	SYM	200	20	MED	HARD	4	13	3
16	SEP	50	20	EASY	MED	0	9	11
16	SEP	50	20	EASY	HARD	1	14	5
16	SEP	50	20	MED	HARD	2	13	5
16	SEP	100	20	EASY	MED	1	7	12
16	SEP	100	20	EASY	HARD	1	16	3
16	SEP	100	20	MED	HARD	1	18	1



Size	Mode	Limit	Runs	P1	P2	P1 Win(s)	P2 Win(s)	Draw(s)
16	SEP	150	20	EASY	MED	2	11	7
16	SEP	150	20	EASY	HARD	0	11	9
16	SEP	150	20	MED	HARD	1	16	3
16	SEP	200	20	EASY	MED	1	12	7
16	SEP	200	20	EASY	HARD	0	14	6
16	SEP	200	20	MED	HARD	4	13	3
16	NON	50	20	EASY	MED	0	20	0
16	NON	50	20	EASY	HARD	0	20	0
16	NON	50	20	MED	HARD	4	15	1
16	NON	100	20	EASY	MED	0	19	1
16	NON	100	20	EASY	HARD	0	20	0
16	NON	100	20	MED	HARD	2	18	0
16	NON	150	20	EASY	MED	0	20	0
16	NON	150	20	EASY	HARD	0	20	0
16	NON	150	20	MED	HARD	2	18	0
16	NON	200	20	EASY	MED	0	20	0
16	NON	200	20	EASY	HARD	0	20	0
16	NON	200	20	MED	HARD	4	16	0
24	SYM	50	20	EASY	MED	0	17	3
24	SYM	50	20	EASY	HARD	0	16	4
24	SYM	50	20	MED	HARD	5	6	9
24	SYM	100	20	EASY	MED	0	9	11
24	SYM	100	20	EASY	HARD	0	17	3
24	SYM	100	20	MED	HARD	4	11	5
24	SYM	150	20	EASY	MED	0	15	5
24	SYM	150	20	EASY	HARD	1	17	2
24	SYM	150	20	MED	HARD	7	11	2
24	SYM	200	20	EASY	MED	0	12	8
24	SYM	200	20	EASY	HARD	0	17	3
24	SYM	200	20	MED	HARD	6	10	4
24	SEP	50	20	EASY	MED	2	12	6
24	SEP	50	20	EASY	HARD	0	12	8
24	SEP	50	20	MED	HARD	2	9	9
24	SEP	100	20	EASY	MED	2	11	7
24	SEP	100	20	EASY	HARD	0	13	7
24	SEP	100	20	MED	HARD	3	13	4
24	SEP	150	20	EASY	MED	1	10	9
24	SEP	150	20	EASY	HARD	2	13	5
24	SEP	150	20	MED	HARD	1	14	5
24	SEP	200	20	EASY	MED	1	10	9
24	SEP	200	20	EASY	HARD	2	16	2
24	SEP	200	20	MED	HARD	3	11	6
24	NON	50	20	EASY	MED	0	19	1
24	NON	50	20	EASY	HARD	0	20	0
24	NON	50	20	MED	HARD	7	13	0
24	NON	100	20	EASY	MED	0	20	0
24	NON	100	20	EASY	HARD	0	20	0



Size	Mode	Limit	Runs	P1	P2	P1 Win(s)	P2 Win(s)	Draw(s)
24	NON	100	20	MED	HARD	5	15	0
24	NON	150	20	EASY	MED	0	20	0
24	NON	150	20	EASY	HARD	0	20	0
24	NON	150	20	MED	HARD	2	18	0
24	NON	200	20	EASY	MED	0	19	1
24	NON	200	20	EASY	HARD	0	20	0
24	NON	200	20	MED	HARD	6	13	1

A.3 Chi tiết mã nguồn

A.3.1 Hướng dẫn triển khai

Trước hết, cần cài đặt các thư viện và môi trường cần thiết để chạy ứng dụng. Toàn bộ các thư viện phụ thuộc đã được liệt kê trong tệp `requirements.txt`. Có thể cài đặt bằng lệnh:

```
pip install -r requirements.txt
```

Ngoài ra, trước khi chạy chương trình chính, cần đảm bảo rằng tất cả các tệp mã nguồn `.cpp` đã được biên dịch thành các tệp thực thi `.exe`. Điều này là cần thiết vì hệ thống của trò chơi sử dụng cơ chế gọi tiến trình ngoài (subprocess) để thực thi các module được viết bằng C++, bao gồm module sinh map và các module của các bot.

Sau khi hoàn tất quá trình cài đặt và biên dịch, có thể khởi chạy trò chơi bằng lệnh:

```
python game.py
```

Nếu quá trình triển khai thành công, giao diện trò chơi sẽ được khởi tạo và người chơi có thể lựa chọn các chế độ chơi tương ứng từ màn hình menu chính.

A.3.2 Cấu trúc thư mục

```
-- requirements.txt
-- game.py
-- simulator.py
-- core/
|   |-- config.py
|   |-- game_screen.py
|   |-- init_screen.py
|   |-- menu_screen.py
|   |-- tutorial_screen.py
|   |-- ui_button.py
|   |-- utils.py
-- setup/
|   |-- gameSetup.py
|   |-- mapGen (.cpp and .exe)
-- interactor/
|   |-- bots (.cpp and .exe)
|   |-- interactor.py
-- graphics/
|   |-- silkscreen.ttf
|   |-- player-1.png
|   |-- player-2.png
```



```
|-- box.png  
|-- tile.png  
|-- wall.png  
|-- portal-1.png  
|-- portal-2.png
```

A.3.3 Docstring

Dưới đây là danh sách docstring của toàn bộ ứng dụng.

```
== game.py ==
```

Function `recreate_init_screen`:

Reinstantiate the initialization screen while capturing previous settings.

Clones player toggle flags, bot difficulty rankings, move constraints, and map generation parameters seamlessly from the obsolete menu instance so configuration profiles aren't lost upon returning from other interfaces.

Args:

`old_init`: The exact prior 'InitScreen' object serving as the config template.

Returns:

`InitScreen`: A freshly constructed screen instance pre-filled with historic values.

Function `main`:

Main game loop managing state transitions between menu, init, and game screens.

```
== core\game_screen.py ==
```

Class `GameScreen`:

Main game screen handling gameplay, rendering, and turn resolution.

Function `__init__`:

Initialize game screen with rendering fonts, buttons, and game state.

Function `draw_grid`:

Draw game grid with walls and floor tiles.

Function `draw_players`:

Draw player sprites at their current positions.

Function `draw_boxes_portals`:

Draw boxes and portal targets at their positions.

Function `draw_score`:

Draw player scores in top corners.

Function `draw_ready_state`:

Draw ready indicators when players have submitted their moves.

Function `_capture_state`:

Capture current game state for undo functionality.



Returns:
Dictionary containing positions , boxes , scores , and move history

Function _restore_state:
Restore game state from captured snapshot.

Args:
state: Previously captured game state dictionary

Function undo_turn:
Undo the last turn by restoring previous game state.

Function reset_game:
Reset game while keeping current map.

Function new_game:
Start completely new game with fresh map.

Function _in_grid:
Check if coordinates are within grid bounds.

Function _check_shift:
Check if boxes can be shifted in a direction and return indices.

Args:
x, y: Starting position
dx, dy: Direction deltas

Returns:
List of box indices that can be shifted , empty if blocked

Function _change_position:
Legacy method: attempt to change player position with box interaction.

Args:
curPos: Current position tuple
dx, dy: Direction deltas

Returns:
True if movement resulted in action

Function _move:
Legacy method: calculate new position from delta.

Function _attempt_plan:
Plan a single player move on current state.

Args:
curPos: Current player position
dx, dy: Desired movement direction
blocked_cells: Set of forbidden cells

Returns:
Dict with valid flag , new position , and list of box moves

Function _apply_all_box_moves_and_score:
Apply all box moves simultaneously and resolve scoring.

Args:



p1_box_moves: List of box moves by P1
p2_box_moves: List of box moves by P2

Function _move_players:
Execute simultaneous player movement with conflict resolution.

Returns:
Tuple of (p1_moved, p2_moved) booleans

Function _get_key:
Get input from bot AI for specified player.

Args:
player: True for P2, False for P1

Returns:
Direction string (U/D/L/R/S)

Function _resolve_turn_if_ready:
Execute turn resolution when both players have submitted moves.

Function _end_game:
Determine winner and end game.

Function handle_event:
Handle game screen events including input and UI interaction.

Returns:
Navigation signal ("MENU", "INIT") or None

Function draw:
Render complete game screen with UI elements.

== core\init_screen.py ==

Class InitScreen:
Game initialization screen for player/bot selection and settings.

Function __init__:
Initialize init screen with buttons for player modes, difficulty, and move limits.

Function _apply_settings:
Apply selected settings to global game variables.

Function draw:
Render initialization screen with all configuration options.

Function handle_event:
Handle keyboard and mouse events on init screen.

Returns:
"START" to begin game, "BACK" to return to menu, None otherwise

== core\menu_screen.py ==



Class MenuScreen:
Main menu with Play and Tutorial buttons.

Function __init__:
Initialize menu screen with fonts and button layout.

Function draw:
Render menu screen with title and buttons.

Function handle_event:
Handle menu events.

Returns:
"PLAY", "TUTORIAL", or None

== core\tutorial_screen.py ==

Function wrap_text:
Wrap text to fit within a given width.

Class TutorialScreen:
Screen displaying game rules, controls, and specific mechanics.

Function draw:
Render the tutorial screen.

Function handle_event:
Handle interactions for navigating back and scrolling.

== core\ui_button.py ==

Class UIButton:
Simple clickable button UI element with hover effects.

Function __init__:
Initialize button.

Args:
rect: Button rectangle
text: Button label text

Function draw:
Draw button on surface with optional hover effect.

Args:
surface: Pygame surface to draw on
font: Font for button text
hovered: Whether button is currently hovered

Function hit:
Check if position collides with button.

== core\utils.py ==



Function `_draw_text`:
Render and blit text at specified position.

Function `reset_game_state`:
Reset game state including map, scores, and move history.

Args:
 generate: If True, generate new map; if False, reuse current map

== interactor\beam_search.cpp ==

```
/**
 * @brief Converts a character representation of a move into an integer
 *        ID.
 * @param c The physical move character ('U', 'L', 'D', 'R', or anything
 *        else for stay).
 * @return Integer ID from 0 to 4.
 */

/**
 * @brief Zobrist hashing implementation for state tracking.
 *
 * Generates and stores 64-bit random integers for cell states,
 * player positions, and scores to quickly compute distinct hashes
 * for explored game states, preventing redundant evaluations.
 */

/**
 * @brief Represents a single possible state of the game during the search.
 *
 * Tracks dynamic properties such as player position, box locations,
 * the sequence of actions, the scoring delta, and the Zobrist hash
 * associated with this specific configuration.
 */

/**
 * @class BeamSearch
 * @brief Solves multiplayer Sokoban by strategically choosing the most
 *        value move.
 *
 * This class implements a heuristic-based solver for Sokoban that:
 * - Uses Beam Search to find optimal move.
 * - Operates within a strict time limit (450ms) for partial solutions
 *
 * @param board Initial game board with player ('a'), goal ('A'), boxes ('X'),
 *        walls ('#')
 */

/**
 * @brief Modifies a state in-place by applying a move action.
 *
 * Handles movement, static board boundaries, collision with walls
 * or opponents,
 * and box pushing (including chain-pushes and scoring goals).
 *
 * @param s The current state to modify.
 * @param d The action direction ID to apply.
 * @return True if the move is valid and successfully applied, False
```



```
        otherwise .  
    */  
  
=====
```

```
===== interactor\beam_search_aggressive.cpp =====
```

```
/**  
 * @brief Zobrist hashing implementation for state tracking.  
 *  
 * Generates and stores 64-bit random integers for cell states ,  
 * player positions , and scores to quickly compute distinct hashes  
 * for explored game states , preventing redundant evaluations.  
 */  
  
/**  
 * @brief Static board layout and initial game elements.  
 *  
 * Stores the dimensions , static terrain (walls and goals),  
 * initial player positions ('a' and 'b'), and initial locations  
 * of all pushable boxes.  
 */  
  
/**  
 * @brief Represents a single possible state of the game during the search.  
 *  
 * Tracks dynamic properties such as player position , box location bitsets ,  
 * the sequence of actions , the scoring delta , and the Zobrist hash  
 * associated with this specific configuration.  
 */  
  
/**  
 * @brief Beam search engine for determining the optimal action.  
 *  
 * Conducts a beam search over upcoming turns. It computes distance  
 * fields globally , applies simulated actions (including box pushing),  
 * evaluates board states using a heuristic , and returns the first step  
 * of the most promising action path found.  
 */  
  
/**  
 * @brief Main entry point for the competitive bot.  
 *  
 * Reads the game state from standard input , which includes :  
 * – Grid size (S) , current turn (Tcur) , and turn limit (Ttotal)  
 * – The map elements represented as characters  
 * – The history of previous actions  
 * – The current scores  
 *  
 * It maps characters like 'a' (player) , 'b' (opponent) , 'X' (boxes) ,  
 * and endpoints to initialize the BoardInfo configuration . Then , it  
 * invokes  
 * the Solver to find the best response and prints the corresponding  
 * action character ('U' , 'D' , 'L' , 'R' , or 'S') to stdout .  
 *  
 * @return 0 on successful completion .  
 */
```



== interactor\interactor.py ==

Function interact:

Execute a game interactor program with game state input.

Runs an executable file with formatted game state information via stdin.
The input dictionary is converted into a multi-line string format expected by the interactor program.

Args:

- filename (str): Name of the executable file to run (relative to interactor folder).
- input (dict): Dictionary containing game state with keys:
 - 'size': Map size
 - 'cur': Current turn number
 - 'T': Total turns
 - 'maze': 2D list representing the game maze
 - 'playerHist': Player move history
 - 'oppHist': Opponent move history
 - 'playerDidMove': Whether player moved (per turn)
 - 'oppDidMove': Whether opponent moved (per turn)
 - 'playerScore': Player's current score
 - 'oppScore': Opponent's current score

Returns:

str: The stdout output from the executed program.

Function compile_cpp_files:

Compile all C++ files in the interactor directory.

Searches the directory containing this script for all .cpp files and compiles each one using g++ to create corresponding executable files. Each compiled executable has the same name as the source file without the .cpp extension.

Raises:

subprocess.CalledProcessError: If compilation of any C++ file fails.

== interactor\lftroq.cpp ==

```
/**
 * @class SokobanSolver
 * @brief Solves single-player Sokoban puzzle by strategically moving boxes
 *        to goals.
 *
 * This class implements a heuristic-based solver for Sokoban that:
 * – Identifies and prioritizes boxes based on proximity to player and goal
 * – Uses BFS to find paths for pushing boxes to target positions
 * – Operates within a strict time limit (450ms) for partial solutions
 *
 * @param board Initial game board with player ('a'), goal ('A'), boxes ('X'), walls ('#')
 */

/**
 * @method SokobanSolver::solve()
 * @brief Finds a partial or complete solution within TIME_LIMIT_MS
 *
 * .
```



```
*
* Algorithm:
* 1. Selects the most promising box using heuristic evaluation
* 2. Checks if box can be pushed toward goal from reachable
   positions
* 3. Executes push sequence if path exists
* 4. Returns after first successful move or time limit exceeded
*
* @return String of moves (U/L/D/R) representing player actions ,
   or empty if no solution found
* @time O(N^4) per move due to BFS operations
*/

/**
* @method SokobanSolver::heuristicEvaluation(pair<int,int> boxes)
* @brief Evaluates priority of a box for pushing.
*
* Uses weighted combination: 2 * (negative player distance) + (
   negative goal distance)
* Prioritizes boxes close to goal while maintaining player
   accessibility.
*
* @param boxes Target box coordinates
* @return Double value representing box priority (higher = better)
*/

/**
* @method SokobanSolver::bfsReach(pair<int,int> player , pair<int ,
   int> box, pair<int,int> goal)
* @brief BFS pathfinding that treats box as obstacle.
*
* Finds shortest path from player to goal without pushing the
   specified box.
* Used to position player behind boxes for pushing.
*
* @param player Starting position
* @param box Obstacle position
* @param goal Target position
* @return Path string or "#" if unreachable
*/

/**
* @method SokobanSolver::bfsBox(pair<int,int> player , pair<int,int>
   > box, pair<int,int> goal)
* @brief Dijkstra's algorithm to push box to goal position.
*
* Explores states combining box position and push direction.
* Two transition cases:
* 1. Change push direction: reposition player and continue
* 2. Continue pushing: advance box in same direction
*
* @param player Current player position
* @param box Current box position
* @param goal Target box position
* @return Complete move sequence to push box to goal , or "#" if
   impossible
*/
```



```
== interactor\lftroq_v2.cpp ==

/**
 * @class SokobanSolver
 * @brief Solves single-player Sokoban puzzle by strategically moving boxes
 *        to goals.
 *
 * This class implements a heuristic-based solver for Sokoban that:
 * - Identifies and prioritizes boxes based on proximity to player and goal
 * - Uses BFS to find paths for pushing boxes to target positions
 * - Operates within a strict time limit (450ms) for partial solutions
 *
 * @param board Initial game board with player ('a'), goal ('A'), boxes ('X'),
 *        walls ('#')
 */

/**
 * @method SokobanSolver::solve(bool InvisibleBoxes = false)
 * @brief Finds a partial or complete solution within TIME_LIMIT_MS
 *
 * @param InvisibleBoxes Consider boxes as empty cells or not
 *
 * Algorithm:
 * 1. Selects the most promising box using heuristic evaluation
 * 2. Checks if box can be pushed toward goal from reachable
 *    positions
 * 3. Executes push sequence if path exists
 * 4. Returns after first successful move or time limit exceeded
 *
 * @return String of moves (U/L/D/R) representing player actions,
 *         or empty if no solution found
 * @time O(N^4) per move due to BFS operations
 */

/**
 * @method SokobanSolver::heuristicEvaluation(pair<int,int> boxes)
 * @brief Evaluates priority of a box for pushing.
 *
 * Uses weighted combination: 3 * (negative player distance) + (
 *   negative goal distance)
 * Prioritizes boxes close to goal while maintaining player
 *   accessibility.
 *
 * @param boxes Target box coordinates
 * @return Double value representing box priority (higher = better)
 */

/**
 * @method SokobanSolver::bfsReach(pair<int,int> player, pair<int,
 *   int> box, pair<int,int> goal)
 * @brief BFS pathfinding that treats box as obstacle.
 *
 * Finds shortest path from player to goal without pushing the
 *   specified box.
 * Used to position player behind boxes for pushing.
 *
 * @param player Starting position
 * @param box Obstacle position
 * @param goal Target position
 */
```



```
* @return Path string or "#" if unreachable
*/

/**
 * @method SokobanSolver::bfsBox(pair<int,int> player, pair<int,int>
 *   > box, pair<int,int> goal)
 * @brief Dijkstra's algorithm to push box to goal position.
 *
 * Explores states combining box position and push direction.
 * Two transition cases:
 * 1. Change push direction: reposition player and continue
 * 2. Continue pushing: advance box in same direction
 *
 * @param player Current player position
 * @param box Current box position
 * @param goal Target box position
 * @return Complete move sequence to push box to goal, or "#" if
 *   impossible
 */

/**
 * @class AggressivePlayer
 * @brief Defensive strategy when player is ahead in score.
 *
 * Blocks opponent by:
 * - Moving toward opponent's goal location
 * - Positioning near boxes to deny box access
 * - Minimizing opponent's movement options
 *
 * @param board Current game board state
 */

/**
 * @method AggressivePlayer::solve()
 * @brief Computes blocking move when player has score advantage.
 *
 * Tries all 4 directions around opponent goal, selects move that:
 * 1. Minimizes distance to opponent
 * 2. Maximizes distance from opponent to nearest box
 * 3. Is reachable from current player position
 *
 * @return Single move character (U/L/D/R) or 'S' (stay) if no
 *   valid move
 */
```

== setup\gameSetup.py ==

Module:

Game Configuration and Initialization Module

This module initializes Pygame, configures game constants (grid size, tiles, colors), loads graphical assets, and provides the map generation function to load or generate the 2P Sokoban puzzle maps.

Function load_assets:

Load and rescale all graphical assets based on the current tileSize.



This updates the global image objects dynamically whenever the grid size forces a scale readjustment for visual responsiveness.

Function `setMapSize`:

Dynamically reconfigure the game grid and UI parameters.

Automatically scales the 'tileSize' according to the grid dimension 'N' to ensure the entire map elegantly fits within readable boundaries, recalibrates

the overall display coordinates, applies the necessary margin paddings, and prompts the main application window to resize securely.

Args:

`N (int)`: The dimensional size of the map (default 16).

Function `_resolve_mapgen_exe`:

Locate the map generator executable.

Searches for the map generator executable in platform-specific order. First checks for `mapGen.exe` (Windows), then `mapGen` (Linux/Unix).

Args:

`base_dir (str)`: Base directory to search for the executable.

Returns:

`str`: Absolute path to the existing executable, or path to `mapGen` if neither exists (caller responsible for error handling).

Function `getMap`:

Load or generate a game map and initialize game state.

Optionally runs the map generator to create a maze configuration, then parses the `maze.txt` file to populate the game grid and entity positions. Updates global variables for grid, player positions, portals, and boxes.

Supported map symbols:

'#': Wall
'.': Walkable tile
'X': Box
'A': Portal 1
'B': Portal 2
'a': Player 1 starting position
'b': Player 2 starting position

Args:

`generate (bool)`: Whether to run the map generator. Defaults to `False`.

`N (int)`: Map size (typically 16). Defaults to 16.

`MODE (str)`: Generation mode — 'SEP' (separate), 'DEF' (default), or 'SYM' (symmetric).

Defaults to 'SEP'.

Raises:

`FileNotFoundError`: If `maze.txt` cannot be found.

`IOError`: If `maze.txt` cannot be read.

== `setup\mapGen.cpp` ==



```
/**
 * @file mapGen.cpp
 * @brief Maze generation utility for game map creation
 *
 * This module generates various types of maze layouts with obstacles,
 * portals, boxes, and player/goal positions. It supports three generation
 * modes: standard maze, separated maze (with middle wall), and symmetric
 * maze.
 *
 * @details
 * - Maze cells are encoded as integers:
 *   0 = walkable path, 1 = wall, 2 = box, 4 = portal A, 5 = portal B,
 *   6 = player start position, 7 = goal position
 * - Uses weighted random generation for varied obstacle placement
 * - Supports multiple maze generation strategies
 *
 * @author lftroq
 * @version 1.0
 */

/**
 * @brief Generates a random integer within a given range
 * @param l Lower bound (inclusive)
 * @param r Upper bound (inclusive)
 * @return Random integer between l and r
 */

/**
 * @brief Generates a weighted random integer
 * @param l Lower bound (inclusive)
 * @param r Upper bound (inclusive)
 * @param t Weight parameter (negative = minimum bias, positive = maximum
 *   bias)
 * @return Weighted random integer
 */

/**
 * @brief Generates blockages/obstacles in the maze
 * @param maze Input maze grid
 * @param num_b Number of blockages to generate
 * @param size_b Maximum size of blockages
 * @param size_w Weighting factor for blockage size distribution
 * @return Maze with generated blockages
 */

/**
 * @brief Places portals at random walkable positions
 * @param maze Input maze grid
 * @param mask Bitmask to control which portals to place (bit 0: portal A,
 *   bit 1: portal B)
 * @return Maze with placed portals
 */

/**
 * @brief Randomly places boxes in the maze
 * @param maze Input maze grid
 * @param num_box Number of boxes to place
 * @return Maze with placed boxes
 */
```



```
/**
 * @brief Places player and goal positions in the maze
 * @param maze Input maze grid
 * @param mask Bitmask to control positions (bit 0: player 1 start, bit 1:
 *         player 2 start)
 * @return Maze with placed positions
 */

/**
 * @brief Generates a no-obstacles maze
 * @return Complete maze grid of size NxN
 */

/**
 * @brief Generates a separated maze with a wall dividing two sections
 * @details Creates two symmetric sections separated by a wall row
 * @return Maze grid with separation
 */

/**
 * @brief Generates a vertically symmetric maze
 * @details Upper and lower halves are mirror images with swapped portal/
 *         position pairs
 * @return Vertically symmetric maze grid
 */

/**
 * @brief Converts integer cell encoding to character representation
 * @param x Cell value (0–7)
 * @return Character representation of cell type
 */

/**
 * @brief Outputs maze to file in text format
 * @param maze Maze grid to write
 * @details Writes maze.txt with character-encoded cells
 */
```
